

Confinamiento de varianza en agentes LLM

Un plano de control determinista y plástico sobre un sensor estocástico

Borrador 2.0 — 2026-08-31 Autor: Ariel Edgardo Levy **Estado:** borrador de trabajo. Los dos teoremas están verificados por máquina; toda afirmación empírica lleva su tamaño de muestra y su piso de ruido, y §1.3 declara el alcance. Destino: arXiv cs.LG (primario), cs.AI (cross-list).

Ésta es la única versión mantenida. La redacción inglesa quedó congelada en `historico/paper-en-congelado.md`, anterior a este borrador y superada por él.

Resumen

Un **arnés** de agentes es la estructura de control que envuelve a un LLM y decide qué llamada viene después: un bucle de razonamiento, una descomposición en sub-tareas, un grafo de verificar-replanificar. La práctica moderna de arneses arrastra una deficiencia de diseño: **hereda las propiedades probabilísticas del componente que envuelve**. El arnés está hecho de llamadas al modelo, así que toda propiedad que se le quiera exigir al sistema —reproducibilidad, trazabilidad, cobertura— queda condicionada a la salida de algo que no la tiene.

El caso más claro son las guardas. Un verificador de alucinación implementado como otra llamada al modelo **padece exactamente la deficiencia que vino a corregir**: verificar con el mismo material del que se desconfía no produce una garantía, produce una **segunda estimación correlacionada con la primera**. La forma general, y es la premisa de este trabajo:

Apilar instancias del componente puede subir una **probabilidad**; no puede producir una **propiedad**. §7.5 mide las dos mitades de esa frase: el acuerdo entre paradigmas predice la corrección con precisión total a partir de cuatro coincidencias —eso es la probabilidad— y aun así no habilita una acción irreversible, porque mueve credencia y nunca procedencia.

Así que la garantía tiene que venir de un componente **de otra clase**. Interponemos entre el modelo y el flujo un **motor determinista y plástico** que decide sobre creencias tipadas, y le dejamos al modelo un solo papel: se le pregunta qué dice el material y lo que devuelve se tipa como proposiciones —«esta cuenta figura a nombre de X»— que entran a una base de creencias, cada una con la unidad de la que salió. De ahí que se lo llame **sensor**, y **estocástico** porque la misma pregunta sobre el mismo material puede devolver otra cosa. El motor separa dos funciones que la práctica corriente colapsa:

$\text{decisión} = f(\text{creencias})$	f determinista, tipada, auditable
$\text{creencias} = g(\text{mundo}, \text{sensor})$	g estocástica

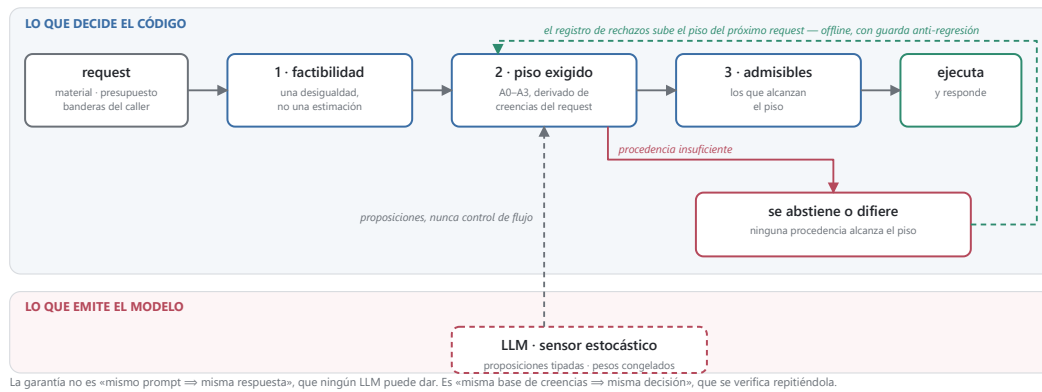
La garantía que eso compra tiene una forma precisa: **misma base de creencias $\Rightarrow$ misma decisión**. Es una propiedad de la decisión, se verifica repitiéndola, y se exhibe junto con la creencia que la disparó. Y **f** es determinista **dentro** de un request y plástica **entre** requests: lo que aprende son **políticas**, que consolidan offline con guarda anti-regresión en un artefacto firmado y se ejecutan como código (§6.3). **El sensor tiene los pesos congelados; el que aprende es el sistema**, porque la base de creencias acumula el registro de sus propias decisiones y de ahí se destila la política. Las reglas escritas a mano dan determinismo; un andamiaje entrenado extremo a extremo da aprendizaje. **El motor da los dos a la vez, y ahí está la novedad: las garantías del sistema vienen del envoltorio y no de lo envuelto.**

Sobre un corpus de análisis forense de hechos, con doce paradigmas medidos bajo condiciones idénticas y sin juez LLM, contribuimos:

1. **Confinamiento de varianza, definido y medido.** Un agente confina su varianza cuando ninguna ramificación de su trayectoria depende de la salida del sensor; la Proposición 5.4 prueba que entonces toda discrepancia entre réplicas tiene un nodo responsable, y por eso es depurable. Empíricamente: entre el **17% y el 34%** de las celdas devuelven resultados distintos a temperatura cero con semilla fija, y absorber ramificaciones en el motor lleva un paradigma de **0,33 a 0,89** en la celda más difícil haciendo coincidir sus réplicas (§5.0, §7.2).
 1. **Un verificador sin oráculo ni juez.** El acuerdo entre paradigmas predice la corrección con precisión total a partir de cuatro coincidencias, supera tres controles —incluido uno que sale en contra— y se reproduce sobre una segunda familia de modelo contra un criterio registrado antes de correr (§7.5).
 1. **Un resultado negativo con mecanismo.** La interacción tarea $\times$ paradigma es el ****48%** de la varianza explicada —**39% del total, contando el ruido entre réplicas— y el premio neto de rutear queda entre -0,019 y -0,034 **** en los tres estratos del held-out: **var(γ)** grande no es premio de ruteo grande, porque la interacción vive entre los paradigmas que nadie elegiría (§7.3).
 1. **Capacidades como abstracción aprendible.** Declaradas desde el código, predicen un paradigma nunca visto mejor que la identidad del paradigma, incluso cuando a ésta se le permite ver la respuesta (§7.4).
- Ninguna pieza del motor es nueva: §2.9 tabula las ocho líneas de las que sale y **qué supone cada una sobre su fuente de creencias que un LLM viola**. Ahí está el aporte — sostener esa disciplina cuando la fuente inventa con forma correcta.
- Palabras clave:** confinamiento de varianza, agentes LLM, plano de control, procedencia, predicción selectiva, aprendizaje plástico, capacidades, reproducibilidad, trazabilidad

El contrato de garantía, y qué pasa cuando la evidencia no alcanza

la abstención es una salida, no un fallo · y el rechazo sube el piso para el próximo request, sin tocar un peso



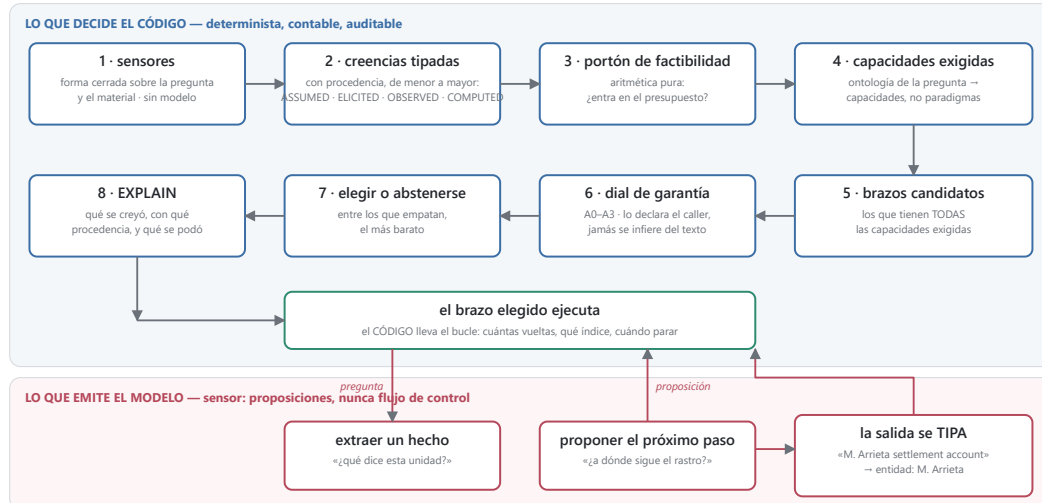
El contrato de garantía, y qué pasa cuando la evidencia no alcanza

Figura 1. El request entra por la izquierda y sale por la derecha, y los cuatro pasos del riel no gastan un token hasta el último. Dos cosas que el resto del paper desarrolla en prosa están acá dibujadas: **la rama de rechazo** —cuando ninguna procedencia alcanza el piso exigido, abstenerse es una salida y no un fallo— y **el lazo**, que es la plasticidad: el registro de rechazos sube el piso del próximo request, offline y con guarda anti-regresión, sin que nadie toque un peso. La flecha punteada que sube del carril de abajo es la única que cruza la frontera, y lleva proposiciones tipadas: nunca control de flujo.

Cómo se decide un request

Cómo se decide un request

la frontera entre los dos carriles es la única decisión de diseño que el registro muestra que importa



Una decisión que cruza al carril de abajo se lleva el determinismo con ella: medido, el ancla de una caminata elegida por el modelo dio $u=1,000 / 0,000 / 0,000$ con la misma huella.

El método determinista: qué decide el código y qué emite el modelo

La figura es el argumento del paper en una imagen. El carril de arriba es determinista y auditable: sensores de forma cerrada, creencias tipadas con procedencia, un compuerta aritmética, las capacidades que la pregunta exige, y recién ahí una elección entre los brazos capaces —o una abstención—. El carril de abajo es el modelo, y **sólo emite proposiciones**: qué dice una unidad, hacia dónde sigue un rastro. Nunca decide cuántas vueltas dar, qué índice usar ni cuándo parar.

Toda decisión que cruza al carril de abajo se lleva el determinismo con ella, y eso está medido, no argumentado. La guarda que cierra el ciclo —tipar la salida del sensor antes de usarla— es del 2026-08-30

y salió de un caso concreto: el modelo emitía 'M. Arrieta settlement account' y esa cola arrastraba la consulta a clasificarse como prosa, mandándola al índice equivocado. **La regla era correcta y la entrada estaba sucia.**

lo que decide el código	lo que emite el modelo
cuántas vueltas dar (del largo declarado en la pregunta)	qué dice esta unidad
qué índice usar (léxico para entidad nombrada, híbrido para prosa)	hacia dónde sigue el rastro
qué unidad es el ancla, y cuál el término de la cadena	qué hecho aporta lo leído
si hay evidencia suficiente para responder, o si hay que abstenerse	la redacción de la respuesta

1. Introducción

1.1 Tres propiedades que un sistema de producción exige

Un *arnés* de agentes es la estructura de control que envuelve al modelo: una llamada única, un bucle de razonamiento, una descomposición, un grafo de verificar-replanificar. Se elige una vez en tiempo de diseño y se congela en el código, y sobre él se construyen sistemas que firman números, disparan acciones y contestan a usuarios. **Cualquier otra capa de un sistema de producción da tres propiedades por sentadas, y en un agente hay que construirlas.**

1. El flujo de control vive en el código. El modelo es aleatorio y va a seguir siéndolo; lo que un arnés decide es **cuánto del sistema depende de esa aleatoriedad**. Delegarle ramificaciones —qué buscar después, cuántas vueltas dar, cuándo parar— vuelve aleatoria la trayectoria misma. Es una decisión de arquitectura, y su precio está medido: entre el **17% y el 34%** de las celdas cambian de resultado entre réplicas a temperatura cero con la misma huella (§7.2.2).

2. Cada valor emitido exhibe de dónde salió. Una respuesta correcta y una inventada llegan con la misma cara, así que la fuente tiene que viajar con el valor. Con una base de creencias tipada se puede exigir que un número esté implicado por evidencia de cierto nivel; el sustituto es la confianza en que «el modelo suele acertar».

3. El sistema puede callarse. En la celda más difícil del corpus, el paradigma que gana lo hace con **12 correctas, 3 abstenciones y cero respuestas equivocadas**; los demás contestan siempre y devuelven un número plausible y falso (§7.2.3). El banco puntúa las dos con **0,000**, que es correcto para medir utilidad y ciego justo sobre el eje que producción necesita.

La tesis es que las tres se consiguen en el mismo lugar: moviendo decisiones de flujo de control desde el modelo hacia el código, sobre señales del entorno que son contables y verificables. Este trabajo las construye, las mide y dice qué cuestan.

1.1.1 La instrucción llega hasta la probabilidad, y una norma pide una propiedad

Hasta ahora el comportamiento determinista se buscó **instruyendo al modelo**: «citá siempre», «si no sabés, decilo», «no inventes números». Los modelos siguen instrucciones cada vez mejor y cada generación se acerca más. Pero una instrucción es un **prior sobre la distribución de salidas**, no una restricción sobre la muestra: seguirla mejor sube la probabilidad, y el límite de esa sucesión queda del lado abierto.

En un chat la brecha da igual. Una norma pide otra cosa: el Reglamento de IA de la Unión Europea exige trazabilidad y supervisión humana sobre sistemas de alto riesgo; el artículo 22 del RGPD condiciona las decisiones automatizadas sobre personas; una auditoría exige exhibir **por qué** se afirmó algo. Los tres piden una **propiedad del sistema**, y eso se consigue con una restricción, no con un pedido. **Este trabajo reemplaza la instrucción por una restricción fuera del modelo.**

1.1.2 Una expectativa de la literatura que este registro no confirma

La motivación habitual para trabajar sobre arneses es que **elegir el paradigma por tarea paga**: la selección oráculo por tarea supera al mejor paradigma fijo por **17,1pp** sobre seis paradigmas, cuatro modelos frontera y diez benchmarks [Select-then-Solve, arXiv:2604.06753]. El mismo trabajo mide que el premio no se cobra —un ruteador sobre embeddings recupera un cuarto de la brecha, el auto-ruteo zero-shot recupera valor *negativo*—, lo que invita a concluir que hacen falta mejores selectores.

Este registro dice otra cosa, y es un resultado. El premio **neto** del ruteo por calidad es **-0,008** contra el mejor fijo, y §7.3 da el mecanismo: la interacción tarea×paradigma existe y es enorme —48% de la varianza— pero **vive entre los paradigmas que nadie elegiría**, y restringida a los que competirían se desvanece. No hacen falta mejores selectores: hace falta medir el premio con el estimador correcto antes de construir uno.

1.2 Tres afirmaciones que preceden al problema de aprendizaje

La factibilidad es aritmética, y es gratis. Antes de preguntar cuál topología es *mejor*, se puede preguntar cuál puede *correr*. Esa pregunta se responde con cantidades que la tarea ya declara. Una política aprendida que gasta episodios descubriendo que map-reduce pierde en tareas de 500 unidades está aprendiendo aritmética por el camino difícil; el tope era computable antes del primer token.

La selección paga sólo bajo una condición precisa. Un ruteador que debe elegir siempre no tiene ningún grado de libertad sobre su tasa de falsos positivos y por lo tanto paga cada error de ruteo. Formalizamos cuándo la selección le gana a un fallback fijo y mostramos que el punto de operación óptimo generalmente implica **abstenerse en la mayoría de las solicitudes** — lo que no es un ruteador más débil sino un objetivo distinto.

Mucho de lo que se le atribuye a la topología es atribuible a la superficie. Entre el paradigma más caro y el más barato del plantel hay un factor **11× de costo** y **0,17** de utilidad: en calidad se separan por centésimas y en lo que cuestan, por órdenes de magnitud (§7.1). Y lo que gobierna esa diferencia no es la estructura de control sino **cuánto material arrastra cada uno al prompt** — el costo es de 98,6% a 100,0% de entrada en todos ellos. Un benchmark que no reporta la calidad de su superficie de herramientas no está comparando topologías: está comparando un retriever envuelto de doce maneras.

1.3 Alcance de las afirmaciones

Establecido: la teoría (§5), verificada por máquina contra distribuciones sintéticas de respuesta conocida; una compuerta de factibilidad (§4) validada sobre cuatro escalas de corpus; un generador de corpus cuyo ground truth se re-deriva de forma independiente a partir de los documentos (§6).

Medido: todo §7, sobre una campaña homogénea de **78 tareas × 12 paradigmas × 3 réplicas** —121,4M tokens, cero errores de infraestructura, sin juez LLM y con el corrector auditado— más un held-out corrido por el mismo camino de código y una réplica del hallazgo de consenso sobre una **segunda familia de modelo**. Cada panel declara su rectángulo y cada comparación su piso de ruido.

1.3.1 El dominio

Lo que se evalúa es capacidad de RAG sobre análisis forense de hechos: un sistema que recibe preguntas del tipo que hace un auditor, un analista de cumplimiento o un investigador sobre un conjunto

documental. Qué cuenta figura a nombre de quién, quién le reporta a quién subiendo N escalones, si existe registro de una transferencia, si una designación sigue vigente, si alguien debía escalar y no lo hizo.

El corpus es heterogéneo por construcción, y sus once modos son los modos de falla de ese dominio, no una muestra de dificultad: hecho único verificable, enumeración independiente, cadena acoplada, agregación con cobertura exhaustiva, horizonte desconocido, acción irreversible, vigencia, plantel declarado, **ausencia**, **presuposición falsa** y escritura compartida. Las dos en negrita son las que un benchmark de extracción no suele tener y las que un sistema forense necesita: la respuesta correcta es que el hecho **no está**, o que la pregunta **da por cierto algo que es falso**.

Sobre eso se apilan tres anchos —5, 20 y 60 unidades— y entidades con variantes de superficie, anáfora y señuelos: el **36,5%** de las menciones son invisibles a una búsqueda del nombre completo y el **100%** de los saltos de cadena exige resolver una variante.

La maquinaria no depende del dominio. Sensor estocástico, motor determinista, procedencia tipada, compuerta aritmética y abstención son propiedades de **cómo se decide**, no de qué se pregunta; §6 las instancia sobre otras tres superficies —datos, acciones, gobierno— con el mismo enunciado sobre otro tipo de proposición. Lo que este registro establece es que la separación **funciona y qué cuesta** en un dominio; que el número transfiera a otro es una predicción, no un resultado.

Fuera de alcance: un selector validado; ningún resultado sobre benchmarks públicos; y ninguna medición fuera del dominio descrito arriba.

2. Trabajo relacionado

2.1 Búsqueda automática de workflows

AFlow reformula la optimización de workflows como búsqueda sobre workflows representados como código, con MCTS sobre operadores [arXiv:2410.10762]; ADAS y GPTSwarm buscan en espacios afines. Estos producen **un** workflow por benchmark, offline. Nuestro interés es la selección por solicitud dentro de un conjunto fijo y — antes de eso — qué miembros del conjunto pueden correr siquiera.

2.2 Selección de paradigma en tiempo de inferencia

Select-then-Solve entrena un ruteador liviano sobre embeddings para elegir un paradigma por tarea [arXiv:2604.06753]. FlowBank construye un portafolio offline y selecciona por consulta [arXiv:2606.11290]. TRACE-Router rutea a granularidad de traza de tarea y no de solicitud [arXiv:2607.22465]. Uno-Orchestra aprende una política conjunta de descomposición y despacho [arXiv:2605.05007].

Todos operan a **cobertura uno**: toda tarea recibe un paradigma. §5.1 sostiene que ésa es la restricción vinculante, más que la capacidad del modelo, y ninguno reporta una curva de riesgo-cobertura. Dos detalles de Select-then-Solve, verificados contra el paper completo y no contra su abstract (2026-08-26): su oráculo es un *máximo empírico sin corregir* por tarea, y sus propias limitaciones anotan que la muestra es fija, sin re-muestreo entre seeds — exactamente el sesgo hacia arriba que el protocolo de piso de ruido (§6.1) existe para descontar. Y la maquinaria de deferral existe al lado, sin haber cruzado: ReDAct difiere decisiones individuales a un modelo más grande sobre un umbral calibrado de incertidumbre [arXiv:2604.07036]; el ruteo por descomposición de incertidumbre unifica abstención y ruteo con garantías distribution-free, para clasificadores [arXiv:2605.07805]; y el meta-ruteo composicional entrena un ruteador interpretable sobre features textuales y nombra un confidence gate con fallback a ruteo estático como trabajo futuro sin evaluar [arXiv:2608.00106]. Nadie aplica abstención a la selección de paradigmas (buscado 2026-08-26); la ventana se está cerrando a la vista.

2.3 Modelos de costo para workflows

GLOW predice performance de workflows agénticos a partir de features de grafo y de lenguaje [arXiv:2512.15751]; Cost-Aware Optimization for Agentic Query Execution hace explícita la analogía con la optimización clásica de consultas [arXiv:2606.03152]. Tomamos la analogía como establecida y no la reclamamos. Nuestro aporte está aguas arriba: un filtro duro de factibilidad que un modelo de costo no reemplaza, porque **un plan infactible no tiene costo**.

2.4 Gobernanza simbólica sobre inferencia probabilística

Es la vecindad más cercana a §6.2 y hay cuatro trabajos dentro. Los dos primeros se leyeron completos el 2026-08-26; la tabla dice qué aporta cada uno y qué de §6.2 queda afuera de su alcance.

Cómo leer la columna «qué no tiene». La primera fila es el trabajo en el que **§6.2 está inspirado**, no un competidor al que se le buscan huecos: esa columna no señala defectos suyos, enumera **dónde se apoya el incremento**. Lo mismo vale para los tres mecanismos de §2.10.

trabajo	qué aporta	qué no tiene
SCL / Soft Symbolic Control [arXiv:2511.17673]	gobernanza sobre inferencia probabilística, partida en <i>Regulation</i> (metaprompt persistente, cuya efectividad el propio paper hace depender de qué tan bien el modelo interprete instrucciones) y <i>Control</i> (runtime determinista sobre el historial del turno: llamadas duplicadas, conteo de errores, profundidad de ciclos) más clasificación de riesgo por acción	opera como un modo global único: no gradúa la garantía por solicitud, no la deriva de creencias sobre la solicitud, no restringe el espacio de planes por nivel, y no calibra la confianza declarada. Sus reglas leen el historial de ejecución, no el contenido proposicional
MINERVA / HADD — Jaime y Errecalde (2026) [Zenodo 10.5281/zenodo.20003407] y su compuerta EVR [Zenodo 10.5281/zenodo.19791686]	el antecedente más cercano, y aporta mecanismo y no vocabulario: LLM confinado a sensor tipado sin control de flujo, cognición determinista como función pura de la base de creencias («mismo estado → misma acción», la forma de garantía que §6.2 adopta), compuerta de admisión en la frontera de percepción, historial append-only reproducible, credencia con calibración adaptativa (EMRE), encuadre Kautz Tipo-2	procedencia como jerarquía tipada con semántica de admisibilidad y no campo de trazabilidad; la inadmisibilidad de lo elicitado para acciones irreversibles (su dispositivo más cercano es confirmación humana: consentimiento, no epistémico); garantía graduada por solicitud (en HADD es uniforme, con un umbral de escalación por tenant como único dial); patrones acotados por nivel (su librería de planes se acota globalmente); calibración medida (EMRE aprende confianza y nunca la mide: sin ECE, sin diagramas de confiabilidad). Selección de paradigma, abstención y deferral quedan fuera
Motores de creencias: Nous [arXiv:2606.22030], MemIR [arXiv:2605.25869], Eywa [arXiv:2605.30771], HEP [arXiv:2607.09195]	acotar la confiabilidad por procedencia del canal; tipar la memoria para impedir colapso de fuentes; promover hechos sólo tras validadores contra evidencia inmutable; hacer auditable la evolución de hipótesis	en HEP toda evidencia validada mueve la creencia por igual, sin jerarquía que filtre la promoción. El jardín de senderos que se bifurcan —agentes que proponen y puntúan sobre los mismos datos— está diagnosticado empíricamente [arXiv:2607.01507] sin mecanismo; la partición proponer/puntuar de §6.3 es uno
Contratos de delegación e identidad atestiguada [arXiv:2603.18043]	el vecino más cercano del lado del ruteo, y empírico: rutear sobre calidad auto-reportada selecciona a los peores delegados y rinde peor que al azar ($\emptyset, 55$ contra $\emptyset, 68$). Su remedio es determinista —contratos que acotan autoridad con objetivos, presupuestos y políticas de falla— más identidad reclamada contra atestiguada, y el brazo	su procedencia es una propiedad del reclamo de calidad de un delegado; el de este trabajo es un orden sobre tipos de evidencia, y ese orden es lo que una regla lee. No reportan retículo, ni piso sobre acciones irreversibles, ni curva riesgo-cobertura: miden exactitud de ruteo

trabajo	qué aporta	qué no tiene
	atestiguado llega a ruteo casi óptimo	

Dos observaciones que salen de la tabla. La primera **corroborar la disciplina del sensor desde una dirección adversarial**: la razón de este trabajo para rechazar la auto-evaluación es epistémica —una afirmación sobre su propia suficiencia no supera **ELICITED**— y la de los contratos es que un delegado tiene incentivo a inflar; las dos llegan a la misma prohibición, y **reclamada / atestiguada** es **ELICITED / OBSERVED** restringido a una proposición.

La segunda **achica lo que se puede afirmar**: «procedencia + ruteo + contratos» ya está ocupado como frase, así que la conjunción se enuncia por lo que excluye. Queda sin antecedente el conjunto de: un **retículo de procedencia sobre evidencia**, un **piso que filtra acciones irreversibles** con él, la abstención tasada como **curva riesgo-cobertura medida**, y el mismo cálculo sobre factibilidad, control y contenido. Cualquier término suelto tiene antecedentes.

2.5 Predicción selectiva y aprender a diferir

Nuestra teoría es una aplicación de un marco establecido. La regla de Chow da el rechazo óptimo; Mozannar y Sontag dan un surrogate consistente para diferir a un experto [PMLR v119]; Verma y Nalisnick agregan deferral one-vs-all calibrado [arXiv:2202.03673]; Mohri y colegas dan formulaciones multi-experto principadas. Contribuimos la aplicación a la selección de paradigmas y los corolarios específicos de §5.1, no el marco.

2.6 Agentes que aprenden sin actualizar pesos

Los enfoques de memoria experiencial — ExpeL, aprendizaje reflexivo experiencial, MemSkill, R²-Mem — acumulan insights, reglas o entradas de memoria. Sleep-time compute corre inferencia en tiempo ocioso y reporta ~1/5 de los tokens en inferencia [Letta]; SCM y trabajo afín agregan consolidación de inspiración biológica [arXiv:2604.20943, arXiv:2605.26099]. Todos consolidan **contenido**. §6.3 consolida la **política de control**, y la afirmación sobrevive una búsqueda fechada (2026-08-26) sólo como conjunción, así que la enunciamos como tal: la política se aprende de los episodios propios del agente, se consolida offline, en un artefacto determinista versionado que se ejecuta fuera del LLM, sobre features estructurales de la tarea. Cada conjunto tiene un vecino fuerte. Trace2Policy destila control desde trazas de *expertos* a bases de reglas que se reinyectan *como texto de prompt* [arXiv:2606.10457]; la compilación declarativa de políticas le da a las políticas de orquestación exactamente la forma de artefacto que queremos — determinista, auditable, versionada — pero *escrita por humanos*, no aprendida [arXiv:2603.27299]; el meta-ruteo composicional aprende un ruteador interpretable offline, sobre features textuales y no estructurales [arXiv:2608.00106]. Ninguno aprende su propia política de control hacia un artefacto así.

2.7 Evaluación y jueces

Deliberadamente **no usamos juez LLM**. Un estudio a gran escala sobre 21 modelos y 541.000 juicios reporta confiabilidad sin validez, y que el acuerdo crudo sobreestima la capacidad discriminativa [arXiv:2606.19544]. Las métricas estilo RAGAS exhiben sesgo de posición, de verbosidad y de auto-preferencia, y la mitigación recomendada es corridas repetidas con inspección de dispersión. Como los

tamaños de efecto medidos son de un dígito de puntos porcentuales y la varianza del juez es del mismo orden — y como el sesgo de verbosidad favorecería sistemáticamente a los paradigmas caros cuyo valor está justamente en cuestión — un juez introduciría **un sesgo alineado con la hipótesis**. §6.1 explica la alternativa.

Sobre atribución de fallos, MemFail aísla los fallos de sistemas de memoria en modos de resumen, almacenamiento, recuperación y razonamiento, y puede atribuir un error a uno de ellos sólo porque las operaciones intermedias quedan registradas [arXiv:2605.26667] — convergente con el requisito de §7.2.3, aunque su atribución corre sobre un juez LLM donde la ésta corre sobre trazas deterministas de herramientas. Su titular es también el de este trabajo en miniatura: escalar las memorias recuperadas o la fuerza del modelo rinde poco y a veces degrada, dependiendo de la tarea.

2.8 Posicionamiento

	cobertura	decisión determinista	auditable	filtro de factibilidad	abstención
AFlow / ADAS / GPTSwarm	offline, un workflow	no	no	no	n/c
Select-then-Solve	1,0	no	no	no	no
FlowBank	1,0	no	no	no	no
TRACE-Router	1,0	no	no	no	no
SCL	modo global	sí	sí	no	no
Este trabajo	selectiva	sí	sí	sí	sí

2.9 El linaje clásico, y qué supuesto rompe un sensor estocástico

Las piezas del motor no son nuevas. Cada una viene de una línea con décadas de trabajo, y cada línea **asume algo sobre su fuente de creencias que un LLM viola**. Ese es el aporte y por eso el linaje va en esta sección y no en una nota: lo que sigue no es una lista de precursores sino el catálogo de supuestos que hubo que reemplazar.

línea	qué aporta	qué asume sobre la fuente	qué rompe un LLM
Revisión de creencias, AGM (Alchourrón, Gärdenfors y Makinson, 1985)	qué se retracta cuando entra evidencia contradictoria	el postulado de éxito : la creencia entrante se acepta	una fuente que puede equivocarse vuelve inadmisible aceptar por defecto; de ahí la compuerta de admisión
Mantenimiento de verdad con justificaciones: Doyle (1979), ATMS de de Kleer (1986)	cada creencia exhibe la justificación que la sostiene	que la justificación existe y es recuperable	fabrica justificaciones plausibles y sintácticamente bien formadas
Procedencia de primera clase: Buneman, Khanna y Tan (2001); semianillos de Green, Karvounarakis y Tannen (2007)	de dónde vino un valor, separado de cuánto se le cree	que la procedencia se deriva de la operación, no se declara	no hay operación: hay una emisión, y su procedencia no está en la salida
Arquitectura BDI (Rao y Georgeff, 1995)	deliberación separada de ejecución	sensores que no alucinan — un sensor roto se detecta por inconsistencia	alucina consistentemente, así que la inconsistencia no lo delata
Políticas como datos, aprendidas offline: Soar (Laird, Newell y Rosenbloom, 1987), ACT-R	la política es un artefacto inspeccionable, no un peso	que las condiciones de las reglas son observables	las condiciones interesantes las emite el sensor, y entonces la clave de la política hereda su varianza (§7.3.6)
Plasticidad hebbiana (Hebb, 1949)	una asociación entre pares tiene contenido propio sobre sus marginales	que los eventos asociados son eventos , no descripciones de eventos	la traza es lo que el sensor pidió, no lo que ocurrió
Opción de rechazo (Chow, 1970) y predicción selectiva	cubrir menos a cambio de errar menos	que hay una puntuación de confianza calibrada	la confianza declarada por el modelo no está calibrada, y §7.5 tuvo que construir una
Argumentación abstracta (Dung, 1995)	una conclusión vale si su prueba sobrevive a los ataques disponibles	el marco se define sobre una relación de ataque dada ; instanciarlo exige construirla	el espacio de respuestas plausibles y falsas no es construible, así que la admisión se decide por procedencia y no por supervivencia

La lectura crítica, en una línea por columna. Las tres primeras líneas aportan la maquinaria de creencias y las tres suponen sinceridad, existencia de justificación o derivabilidad de procedencia — y las tres suposiciones caen a la vez cuando la fuente puede inventar con forma correcta. BDI aporta la separación que este trabajo hereda y su detección de sensores rotos no funciona sobre un sensor que alucina de forma consistente. Soar aporta la política como dato y su supuesto de condiciones observables es exactamente lo que §7.3.6 mide que falla. Chow aporta la abstención y su supuesto de confianza calibrada es el que §7.5 reemplaza por una curva medida.

Qué se hizo con cada supuesto roto. La procedencia deja de derivarse y pasa a **declararse y verificarse**: un valor es admisible si el código puede exhibir de qué unidad salió, y si no puede, no se emite (Teorema 2). La confianza deja de leerse del modelo y pasa a **calibrarse contra el registro**. Y las condiciones de la política pasan a exigirse `COMPUTED`, porque una clave elicitada convierte la tabla de decisión en una variable aleatoria (§7.3.6).

2.10 Tres mecanismos que ya están ocupados

Tres trabajos ocupan, cada uno por su lado, mecanismos que esta sección podría reclamar (leídos completos el 2026-08-28).

trabajo	qué ocupa
EnvProbe — <i>Ask the World Before Acting: Budgeted Environment Probing for World-Model Calibration</i> (arXiv 2606.31422)	un operador de sondeo con presupuesto cuyo único propósito es reparar una tabla de creencias estructurada . Es la sonda de §6.2, mecanismo por mecanismo
Kintsugi — <i>Learning Policies by Repairing Executable Knowledge Bases</i> (arXiv 2605.09487)	ediciones a un artefacto ejecutable tipado, filtradas por un verificador , con las fallas diagnosticadas y localizadas en ediciones candidatas. Es la consolidación de §6.3 con su guarda de promoción
ProvenanceGuard — <i>Safeguarding LLM Agents from Misalignment through Provenance Analysis</i> (arXiv 2607.01236)	la desalineación como si una llamada propuesta está sostenida por evidencia trazable en el contexto . Es el piso de procedencia de §5.5 sobre las acciones

Y el área está lo bastante poblada como para tener **survey**: *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents* (arXiv 2606.04990).

Sondeo con presupuesto sobre un estado de creencias tipado, ediciones de política filtradas por verificador, y pisos de procedencia sobre acciones están **cada uno establecido**, y ninguno de los tres es original de este trabajo.

Lo que queda es una **conjunción**, enunciada por lo que excluye: una capa de decisión que (a) elige **qué topología de control de flujo correr**, por request, de un catálogo de ellas — ninguno de los tres rutea entre *paradigmas*; (b) puede **abstenerse**, con la curva riesgo–cobertura reportada en vez de la utilidad de lo que eligió contestar; y (c) poda por **aritmética sobre el presupuesto declarado antes de cualquier inferencia**. Sacando cualquiera de las tres, el resto queda cubierto por el trabajo de arriba.

Y uno de los cuatro nos apoya, desde un lugar al que no llegamos. *Trace2Policy: From Expert Behavior Traces to Self-Evolving Decision Agents* (arXiv 2606.10457) reporta un despliegue en producción de 22 días sobre 3.349 casos resueltos, y encuentra que **a lo largo de cinco escalas de modelo, la varianza atribuible a la versión de la regla supera a la atribuible a la elección de modelo**. Independiente, a escala de producción, y lo más parecido a corroboración externa que tiene esta línea: **la estructura decide más que el modelo**.

3. Preliminares

Una **tarea** t aporta una pregunta, un conjunto de *unidades* (documentos), un presupuesto declarado de tokens, y flags de irreversibilidad y escritura de estado compartido. Un **paradigma** $p \in P$ es una estructura de control que puede llamar herramientas y debe emitir una respuesta. La **calidad** $q(t, p) \in [0, 1]$ es F1 de conjuntos contra un oráculo exacto. El **costo** $c(t, p)$ es el total de tokens sobre cada llamada que el paradigma hace.

El **mejor paradigma fijo** es $p^* = \text{argmax}_p E_t[u(t, p)]$. El **oráculo** es $E_t[\max_p u(t, p)]$, y la **brecha del oráculo** es su diferencia. La utilidad es calidad neta de una preferencia de costo:

$$u(t, p) = q(t, p) - \lambda \cdot (c(t, p) / \min_{\{p'\}} c(t, p') - 1)$$

Normalizar por el paradigma más barato en esa tarea hace a λ interpretable — la calidad que uno está dispuesto a cambiar por un múltiplo extra del costo mínimo — y $\lambda=0$ recupera calidad pura exactamente.

No se elige ningún λ : la calidad y el costo crudos se guardan sin modificar y el trade-off se aplica en tiempo de análisis, de modo que los resultados se reportan como función de λ y no bajo un supuesto sobre λ .

El plantel son doce paradigmas, y se agrupan por **de qué es función su costo** —que es lo que predice quién sobrevive cuando el material crece— y no por su nombre de familia:

ley de costo	paradigmas	qué los define
estructural	<code>direct</code> , <code>rewoo</code> , <code>graph_traverse</code> , <code>extract_compute</code> , <code>streaming_scan</code>	número fijo de llamadas, pase lo que pase
por vueltas	<code>react</code> , <code>reflection</code> , <code>dag_strategy</code> , <code>supervisor</code> , <code>gist_reader</code> , <code>pointer_chase</code>	el costo escala con cuántas veces vuelven al modelo
por alcance	<code>handoff</code>	el costo escala con cuánto material hay

Cinco existen para cotejar contra grillas publicadas. Los otros siete entran cada uno contra un modo de falla concreto: `rewoo` porque planifica todo de antemano y su costo no depende del alcance; `dag_strategy` porque una comparación que omite la topología más elaborada disponible está sesgada a favor de las simples; `handoff` y `supervisor` porque reparten alcance entre sub-agentes de dos maneras distintas; `gist_reader`, `pointer_chase` y `graph_traverse` porque cada uno interpone una representación intermedia más chica que el material —un resumen, un puntero, un índice de entidades— y §7.2.1 mide qué cuesta eso.

La ingeniería de prompts no es un paradigma y no está en el plantel de decisión. Un brazo que sólo cambia el fraseo quedó como control nulo: en toda celda medida da la misma utilidad que la llamada directa y nunca cuesta menos. Los paradigmas se distinguen por **estructura de control de flujo**, jamás por redacción, y esa regla es lo que hace que las mejoras de §7.2.4 sean transferibles en vez de anecdóticas.

4. La factibilidad es aritmética

Algoritmo 3. La compuerta, entera. No hay ninguna llamada al modelo y no hay ningún parámetro aprendido: es una desigualdad sobre cantidades que la tarea ya declara.

ALGORITMO 3 Portón de factibilidad

Entrada : tarea t con unidades U , presupuesto B tokens, plantel P

Salida : $P' \subseteq P$, los que PUEDEN correr

```
1   $N \leftarrow |U|$                                 ▷ cardinalidad
2   $C \leftarrow \sum_{u \in U} \text{tokens}(u)$           ▷ contenido total
3   $A \leftarrow B - \text{reserva\_de\_conversación}(t)$     ▷ lo disponible de verdad
4   $P' \leftarrow \emptyset$ 
5  for cada  $p \in P$  do
6    según ley_de_costo( $p$ ):
7      alcance      :  $\text{coste} \leftarrow C$ 
8      estructural  :  $\text{coste} \leftarrow C / \text{ramas}(p)$ 
9      vueltas      :  $\text{coste} \leftarrow \text{unidad\_máxima}(U) \cdot \text{techo\_de\_llamadas}(p)$ 
10   if  $\text{coste} \leq A$  then  $P' \leftarrow P' \cup \{p\}$ 
11  return  $P'$ 
```

La línea 6 es la que separa dos modos de falla que se confunden de rutina: a un paradigma que reparte por unidad lo acota N y no C , y a uno que lee todo lo acota C y no N . Un corpus con pocas unidades enormes poda a los primeros; uno con muchas unidades chicas, a los segundos. Sin la línea 6 los dos casos se reportan como «no alcanzó el contexto».

Los corpus de esta sección son la escalera de escala, no el corpus de medición. `gold_v2`, `gold_wide` y `gold_deep` existen para barrer cuatro órdenes de magnitud de material —16k a 1,27M tokens— y mostrar que la compuerta de factibilidad se comporta como su aritmética predice en los cuatro. Los resultados de §7 corren sobre otro corpus, con entidades reales y variantes de superficie. **La factibilidad no depende del modelo ni del corpus:** es una desigualdad sobre cantidades declaradas, y por eso se puede validar en un lugar y aplicar en otro.

4.1 El chequeo

Que p pueda correr t depende de cantidades que t ya declara. Con n unidades, contenido C tokens, presupuesto B , y una asignación $A = 0,6 \cdot B$ que deja lugar para la conversación:

paradigma	restricción vinculante	infactible cuando
Direct, CoT	un prompt con todas las unidades	$C > A$
Map-Reduce	una llamada por unidad, más un reduce sobre todos los parciales	$n > 80$ o $n \cdot f > A$
DAG	sub-preguntas $\times$ iteraciones $\times$ replanificaciones	llamadas proyectadas > 200
ReAct, Reflection	tope propio de iteraciones; lee selectivamente	nunca

donde f es el tamaño proyectado de un hallazgo. Sin llamada al modelo, sin estadística, sin aprendizaje.

4.2 Separa dos modos de falla que se confunden de rutina

Aplicado a **168 tareas sobre cinco corpus** del mismo generador, cada tarea con su propio presupuesto declarado:

corpus	tokens/unidad	unidades/tarea	contenido máx	Direct, CoT	Map-Reduce
gold_v2	232	60	16k	39/39	39/39
gold_v3	2.147	60	152k	12/39	39/39
gold_wide	264	500	135k	20/32	18/32
gold_deep	7.161	60	483k	2/26	26/26
gold_xl	2.499	500	1.272k	8/32	18/32

Tareas factibles sobre el total. Los otros cuatro paradigmas son factibles en **168/168**.

Agregado sobre paradigmas, el espacio de planes admisibles se contrae monótonamente con la escala — **273/273 celdas en gold_v2, 186/224 en gold_wide, 134/182 en gold_deep, 162/224 en gold_xl**: de 100% admisible a 72%, íntegramente por aritmética y antes de gastar un token. Un selector que opere sin esta capa tendría que aprender que ese cuarto del espacio es inalcanzable, un episodio fallado a la vez.

El par decisivo es gold_wide contra gold_deep. gold_deep lleva 3,6× más contenido, y Map-Reduce pasa de 18/32 factibles a **26/26** mientras Direct y CoT se derrumban a 2/26. El tamaño total no predice la factibilidad de Map-Reduce; **la cardinalidad sí**. Con 483k tokens en 60 unidades corre, porque nunca las tiene juntas. Con 135k en 500 unidades no: 501 llamadas, y un reduce que concatena 500 hallazgos. Tratar su límite como un límite de contexto lleva a descartarlo exactamente donde funciona.

El mismo tope de acumulación aplica al blackboard del DAG, que renderiza cada hallazgo dentro de cada prompt de sub-agente.

Cuán fuerte muerde la restricción de leer-todo es fácil de subestimar. En gold_deep una tarea de **una sola unidad** es infactible para Direct, porque un documento son 8.075 tokens contra una asignación de 4.800. La restricción no es "el corpus es grande" sino "la pieza más chica que se puede direccionar ya no entra", y ninguna cantidad de lectura selectiva cambia eso para un paradigma cuyo único movimiento es leer.

Una segunda lectura que conviene decir en voz alta. Los cuatro paradigmas selectivos son factibles en las 168 tareas. Es una propiedad real — acotan sus propias iteraciones y leen a demanda — pero también acota lo que esta capa puede hacer. La factibilidad restringe sólo a los paradigmas que retienen material

en contexto o se abren por unidad; **no ofrece ninguna protección contra un paradigma selectivo gastando a través de un corpus de 1,27M tokens**. Esa protección tiene que venir de un presupuesto, no de aritmética sobre el corpus, y §7.2 muestra por qué hace falta: los selectivos son justamente los que varían cincuenta veces en costo.

4.3 Por qué esto va delante del problema de aprendizaje

La factibilidad es determinista, gratis, y aguas arriba de todo lo demás. Poda el espacio de planes antes de cualquier selección, aprendida o no. **El corolario para producción es que saber el largo y declinar no es una degradación** — es la diferencia entre un sistema acotado y uno desbocado.

Medido, y la distinción no es académica. Sobre cuatro celdas de gold_deep, Direct quedó podado en tres y corrió en una, donde sacó 1,000. Registradas como respuestas equivocadas, esas tres dejarían su media en 0,250 — de mejor a peor del plantel, por un artefacto de registro. Registradas como infactibles, el enunciado es el correcto: *el mejor donde puede correr, no disponible donde no*. La misma capa que protege al estudio de una conclusión falsa es la que un sistema en producción necesita para declinar en lugar de fallar.

5. Teoría

5.0 Confinamiento de varianza

El título de este trabajo nombra una propiedad, así que la propiedad necesita definición antes que evidencia.

Definición 5.1 (Trayectoria). Sea q un request y M el material declarado. Una *trayectoria* $T(q, M)$ es la secuencia ordenada de nodos visitados por un agente: cada nodo es un par (unidad leída, llamada emitida).

Definición 5.2 (Punto de ramificación delegado). Un nodo $n_i \in T$ es un *punto de ramificación delegado* si la identidad de n_{i+1} es función de la salida del sensor. Se escribe $d(T) = |\{n_i \in T : n_i \text{ es delegado}\}|$.

La distinción es entre **qué se extrae** de un nodo y **cuál es el nodo siguiente**. Un agente que le pregunta al modelo «¿qué dice esta unidad?» no delega ramificación; uno que le pregunta «¿dónde busco ahora?» sí.

Definición 5.3 (Confinamiento de varianza). La varianza de un agente está *confinada* sobre (q, M) si $d(T(q, M)) = 0$: la secuencia de nodos visitados es una función determinista de q y de M , y la salida del sensor sólo determina el **contenido** atribuido a cada nodo.

Proposición 5.4 (Localización). Si la varianza está confinada, entonces para dos ejecuciones cualesquiera T_1 y T_2 sobre el mismo (q, M) se cumple $T_1 = T_2$ como secuencia de nodos, y toda discrepancia entre sus salidas es atribuible a un nodo identificable.

Demostración. Por inducción sobre la longitud. El nodo inicial es función de q y M por hipótesis. Dado n_i idéntico en ambas ejecuciones, n_{i+1} es función de $(q, M, n_1 \dots n_i)$ y no de la salida del sensor, porque $d(T) = 0$ excluye la dependencia; luego n_{i+1} coincide. Las dos secuencias son iguales. Una discrepancia de salida exige entonces que algún nodo haya recibido contenido distinto, y ese nodo es el testigo. ■

Lo que la proposición compra, y es la razón de ser del confinamiento. Sin él, dos réplicas que difieren pueden diferir porque **leyeron material distinto**, y no hay forma de localizar la causa: la discrepancia se reparte sobre una historia entera. Con él, la discrepancia siempre tiene un nodo responsable — y una discrepancia localizable es depurable, auditable y corregible, mientras una repartida no.

Y lo que NO afirma. El confinamiento no reduce la varianza del sensor ni mejora la calidad de la respuesta: reordena dónde puede manifestarse. La afirmación empírica de §7.2 —que confinar sube la utilidad *además* de estabilizar— es un resultado de ese corpus y no una consecuencia de la Proposición 5.4.

Definición 5.5 (Consecuencia observable). Para k réplicas de una misma celda, $\text{pass}^k = P(\text{todas las } k \text{ réplicas aciertan})$. La Proposición 5.4 implica que con varianza confinada pass^k está acotado inferiormente por pass^1 menos la varianza de extracción de contenido, y no por el producto de las varianzas de ramificación. §7.2.2 mide la brecha en el régimen no confinado.

5.1 El Teorema del Valor de Selección

Es una identidad contable, no un resultado empírico, y se usa como instrumento. No afirma que rutear convenga: descompone exactamente el valor de rutear en términos que se pueden medir por separado, y por eso permite que una evaluación de ruteo sea **falsable**. §5.1.5 desarrolla la distinción. Un teorema presentado como evidencia del sistema sería el error que §2.4 le audita a otros.

Sea p^* el fallback y $p_1 \dots p_k$ los especialistas. Sea $\Delta_j(t) = u(t, p_j) - u(t, p^*)$. Para cada brazo, partir el espacio de tareas **en tres** — la tercera parte no es un tecnicismo, y §5.1.1 muestra qué cuesta colapsarla:

$$\begin{array}{lll} S_+^j = \{ \Delta_j > 0 \} & \text{ganancia estricta} & \pi_j = \Pr[S_+^j] \\ S_0^j = \{ \Delta_j = 0 \} & \text{empate} & \tau_j = \Pr[S_0^j] \\ S_-^j = \{ \Delta_j < 0 \} & \text{pérdida estricta} & \nu_j = \Pr[S_-^j] \end{array}$$

y sean, **condicionadas a lo efectivamente ruteado**,

$$\begin{array}{ll} \alpha_j = \Pr[\text{rutear a } p_j \mid S_+^j] & G_j = E[\Delta_j \mid \text{rutear a } p_j, S_+^j] \\ \beta_j = \Pr[\text{rutear a } p_j \mid S_-^j] & L_j = E[-\Delta_j \mid \text{rutear a } p_j, S_-^j] \end{array}$$

Teorema 1. Para todo $k \geq 1$,

$$V(r) - V(p^*) = \sum_j (\pi_j \cdot \alpha_j \cdot G_j - \nu_j \cdot \beta_j \cdot L_j)$$

exactamente, sin ningún supuesto de independencia. Por lo tanto la selección le gana al mejor paradigma fijo si y sólo si $\sum_j \pi_j \alpha_j G_j > \sum_j \nu_j \beta_j L_j$.

Demostración. $V(r) - V(p^*) = E[\Delta_{\{r(t)\}}(t) \cdot 1\{r(t) \neq p^*\}]$. Descomponiendo por destino queda $\sum_j \Pr[\text{rutear a } p_j] \cdot E[\Delta_j \mid \text{rutear a } p_j]$, y partiendo cada esperanza condicional según el signo de Δ_j : la parte de S_+^j pesa $\pi_j \alpha_j$ con media G_j , la de S_-^j pesa $\nu_j \beta_j$ con media $-L_j$, y S_0^j **aporta exactamente cero** porque ahí $\Delta_j = 0$. ■

Dos decisiones lo vuelven exacto y no aproximado. Condicionar G y L a los subconjuntos *ruteados* elimina todo supuesto de independencia entre dónde vive la ganancia y dónde el ruteador elige ir. Y descomponer **por destino** en vez de por un único «especialista» lo hace valer para un catálogo: con k brazos, equivocarse tiene un *destino*, y mandar una tarea a un brazo apenas peor no es el mismo evento que mandarla al peor de doce.

Corolario 1 (precisión sobre cobertura). Cuando p^* es casi óptimo en regiones amplias, G es chico y L grande, así que la condición exige $\beta \rightarrow 0$ incluso a costa de α . El recall no es el objetivo.

5.1.1 Un empate no es un misruteo

S se define con desigualdad *estricta*, así que los empates quedan afuera. Una versión anterior de este teorema definía β sobre el complemento de S_+ , lo que le cobraba al ruteador haber ruteado sobre un

empate — un acto que no cuesta absolutamente nada. Construido: un ruteador que rutea sólo donde gana o empata, y nunca donde pierde, registra $\beta = 0,714$ sin haber hecho un solo daño.

El producto $\beta \cdot L$ seguía siendo correcto, porque L absorbía el cero. Pero β sola dejaba de ser la *tasa* de misruteo, y el Corolario 2 usa β y L por separado. Definir β sobre S_- restituye la lectura esperada y deja el Teorema 1 intacto: los empates aportan cero a los dos lados.

No es un caso de borde en ningún catálogo donde varios paradigmas resuelven la misma tarea. Acá, un brazo es el más barato al empatar en 46 de 96 celdas.

5.1.2 El umbral de imposibilidad, y contra qué pérdida se mide

Definir, por brazo,

$\bar{L}_j = E[-\Delta_j \mid S_-^j]$	la pérdida media de la TAREA	(no depende del ruteador)
$\rho_j = L_j / \bar{L}_j$	la SELECTIVIDAD DE PÉRDIDA	($\rho_j := 1$ cuando $\beta_j = 0$)

Corolario 2. El ruteador le pierde a siempre-fallback si y sólo si $\sum_j \pi_j \alpha_j G_j < \sum_j v_j \beta_j \rho_j \bar{L}_j$, y con un solo brazo

$$\beta_{\max}(\rho) = \pi \cdot \alpha \cdot G / (v \cdot \rho \cdot \bar{L})$$

ρ	qué ruteador es	qué hace el umbral
$\rho = 1$	ciego a la magnitud de la pérdida — se equivoca de forma representativa	el enunciado clásico, y ahí es exacto
$\rho < 1$	evita las equivocaciones caras	se afloja
$\rho > 1$	anti-calibrado: falla justo donde más duele	se endurece

Por qué el parámetro es necesario y no un refinamiento. Enunciado sólo con la pérdida distribucional, el umbral no es una imposibilidad. Alcanzan tres tareas: una ganancia de $+0,10$ con probabilidad $0,10$, ruteada; una pérdida de $-0,001$ con $0,45$, ruteada; y una de $-1,00$ con $0,45$, no ruteada. Entonces $\beta = 0,500$ supera $\beta_{\max} = 0,0222$ por 23× y el ruteador **igual captura $+0,00955$** . Un ruteador que se equivoca *seguido pero barato* es exactamente lo que produce un ruteo selectivo bien construido.

Tampoco alcanza con sustituir por la pérdida realizada: un ruteador perfecto no realiza pérdida, así que el umbral reportaría infinito y parecería no imponer restricción alguna — la objeción que motivó la forma distribucional en primer lugar. **Las dos objeciones son correctas.** Se disuelven juntas en cuanto la pérdida realizada entra como *factor de la pérdida* y no como *denominador de un umbral*: con $\beta = 0$ el término entero se anula antes de que ρ se consulte.

Corolario 2b. Un ruteador obligado a elegir no controla ni β ni ρ . Uno selectivo controla los dos — y ρ es la palanca más barata: bajar β exige acertar más seguido; bajar ρ sólo exige abstenerse donde la apuesta es cara. Eso le da al Corolario 1 un mecanismo y no sólo una desigualdad, y ρ se recupera de cualquier registro que reporte pérdida potencial y realizada.

5.1.3 Cobertura óptima

Sea $v(c)$ el valor capturado a cobertura c , admitida bajando un umbral de confianza. Entonces $dv/dc = E[\Delta \mid \text{tarea marginal en } c]$, y por lo tanto:

(a) v es unimodal **si y sólo si** $c \mapsto E[\Delta \mid \text{marginal en } c]$ es no creciente — es decir, si y sólo si la señal de confianza ordena las tareas por *ganancia esperada*. **La calibración sola no da eso**. La calibración restringe la probabilidad de ganar; el valor depende de su magnitud. Construido, con $\Pr[\text{acertar} \mid \kappa] = \kappa$ exactamente en cada grupo: tres grupos con $E[\Delta]$ de $+0,008$, $-0,170$ y $+0,593$ a confianzas $0,90$, $0,60$ y $0,30$ producen una curva de valor capturado que **sube, baja y vuelve a subir**, con su óptimo en **cobertura total**.

(b) Sin ningún supuesto: la cobertura óptima es < 1 siempre que alguna tarea con $\Delta_{\{r(t)\}}(t) < 0$ fuera ruteada a cobertura total. Esto es lo que el argumento necesita, y se sigue en una línea — sacar un término negativo aumenta la suma.

Corolario 3. Afirmando (b). Afirmar unimodalidad afirma más de lo que el diseño necesita, y regala un contraejemplo.

5.1.4 La identidad de la brecha del oráculo, y su alcance

Con un solo especialista, la brecha del oráculo iguala $\pi \cdot G$, lo que permite recuperar el β implícito de un ruteador publicado a partir de sus números de portada. **Con k brazos no factoriza**: la brecha es $E[\max_j \Delta_j^+]$, que no es $\pi_j \cdot G_j$ de ningún par fijo. Los 17,1pp reportados para una suite publicada pueden leerse como $\pi \cdot G$ sólo donde ese trabajo reporte un *par*; sobre una grilla, la lectura del β implícito no está disponible.

5.1.5 Qué es el Teorema 1, y qué no es

Es una *identidad*: una descomposición algebraica exacta, verdadera por construcción. **Ninguna medición puede falsarla**, y nada en este paper debe leerse como que la confirmó. Lo empírico es sólo si sus términos satisfacen la desigualdad sobre una distribución dada — una pregunta sobre un ruteador, no sobre el teorema.

El Teorema 1 y los tres corolarios están verificados en `tests/test_science.py` contra distribuciones cuyos términos se conocen por construcción. La identidad se chequeó además sobre 4.000 distribuciones al azar con k de 1 a 4 — 3.269 de ellas con empates — con discrepancia máxima $1,67 \times 10^{-16}$. Los Corolarios 2 y 3 están enunciados arriba en su forma corregida; los contraejemplos que forzaron la corrección se reproducen ahí.

Y la distinción importa en este punto porque los términos nunca se separaron. En los dos corpus held-out el margen de decisión del ruteador fue **0 en todas las tareas**, así que la curva riesgo–cobertura colapsa a un solo punto en el origen: **AURC 0,000** contra un techo de **+0,400**. Un ruteador que nunca se abstiene no tiene α ni β distintos de siempre-fallback, así que la identidad se cumple vacuamente — con los dos términos medidos sobre una cobertura que el ruteador no eligió.

El Teorema 1 aporta la contabilidad; §5.2 aporta las afirmaciones falsables que este registro resolvió — dominancia de cascada y sensibilidad del detector. La rama de selección de la partición **sigue sin ejercitarse** (§8.2).

Una observación a las tres correcciones. π , α y β responden *si* el ruteador acierta; G , L y p responden *cuánto cuesta cuando no*. Cada lugar donde el enunciado anterior falló —el umbral, los empates, la unimodalidad— es un lugar donde esos dos ejes se trataron como uno.

5.2 Dominancia de la cascada, y su corrección medida

Un **ruteador** que se equivoca paga L , una pérdida de calidad: se entrega una respuesta peor y nada la recupera. Una **cascada** que se equivoca paga $\text{cost}(p_1)$, una pérdida de costo: el intento barato se desperdicia y la buena respuesta llega de todas formas tras escalar.

Con sensibilidad de detector s :

$$\begin{aligned}\text{regret}(\text{ruteador}) &= v \cdot \beta \cdot p \cdot L \\ \text{regret}(\text{cascada}) &= E[\text{costo de la escalera}] + (1-s) \cdot L\end{aligned}$$

Una corrección que le debemos a la medición. Un enunciado anterior de este resultado daba el término de costo como $\text{cost}(p_1)$. Es falso. Cuando ningún escalón satisface al detector, la cascada corre la escalera *completa*: en el estudio sintético capturó el 100% de la brecha a **4,4x** el costo del mejor paradigma fijo. El límite es el costo esperado de la escalera, y el patrón requiere un tope de presupuesto.

La sensibilidad del detector es la variable crítica, y un detector débil es catastrófico y no simplemente subóptimo. Con $s = 0,6$ la fracción capturada midió **-5,98**: una falla no detectada deja a la cascada detenida en un escalón barato con una respuesta mala. Es el análogo estructural del Corolario 2 — como un ruteador con β alto pierde, una cascada con s bajo pierde, y pierde más fuerte.

Corolario (la partición por verificabilidad).

$$\begin{aligned}v = 1 \quad (\text{existe un oráculo barato}) &\rightarrow \text{cascada; no hace falta ruteador} \\ v = 0 \quad (\text{no hay oráculo}) &\rightarrow \text{rutear, bajo la disciplina del Teorema 1}\end{aligned}$$

Predecir sólo es necesario donde verificar es imposible. Si esto se sostiene, buena parte de la literatura de ruteo está resolviendo el problema equivocado en el régimen verificable.

Y la partición tiene una consecuencia metodológica filosa que pagamos por aprender. La rama $v = 0$ es la que necesita un ruteador, y es precisamente la rama que un benchmark de coincidencia exacta no puede contener: el gold es lo que hace que corregir no necesite juez, y el gold es un detector. En este registro la confluencia fue literal —un mismo campo servía para las dos cosas— y el resultado es que dos predicciones de ruteo preregistradas, sobre dos corpus held-out independientes, las contestó la regla de cascada mientras la de selección no se evaluó nunca. Los números son reales; lo que miden es la rama $v = 1$.

Separar las dos no es un cambio de calibración. Requiere un corpus que **declare la disponibilidad de detector por tarea** sobre bases independientes de la clave de respuestas —para nosotros, si verificar es más barato que resolver— y mueve la cascada de disparar en 22 de 26 tareas a disparar en 2.

5.3 Soundness del ensamblador

Los tres resultados que siguen son **estructurales**: ninguno depende de un corpus, de un modelo ni de una corrida. Están en esta sección porque la maquinaria que describe §6 existe para hacer posibles enunciados de esta forma, y sin ellos la procedencia es contabilidad — se registra, se muestra, y no compra nada que alguien pueda nombrar.

Sea T una plantilla con ranuras $S = \{s_1 \dots s_n\}$, B una asignación de ranuras a proposiciones, Γ una base de creencias y ϕ un piso de procedencia.

Teorema 2 (soundness del ensamblador). Si `fill(T, B,  $\Gamma$ ,  $\phi$ )` emite una cadena R , entonces para toda ranura $s_i \in S$ existe $\beta_i \in \Gamma$ tal que

1. β_i es la creencia **vigente** sobre la proposición que B asigna a s_i ;
2. `rank(procedencia( $\beta_i$ ))  $\geq$  rank( $\phi$ )`;
3. la subcadena de R en la posición de s_i es exactamente `str(valor( $\beta_i$ ))`.

Y R no contiene ninguna subcadena que provenga de otro lado que no sea T o esos valores.

En una línea: **si el ensamblador emite, todo número que emitió está implicado por la base de creencias al piso pedido.** No «probablemente». No «salvo alucinación».

Demostración. Por construcción, y depende de una línea. `fill` recorre `slots_of(T)` —las ranuras que la plantilla realmente usa, extraídas de ella y no declaradas aparte— y para cada una hace exactamente tres cosas: resuelve la proposición asignada, pide la creencia vigente y compara el rango de procedencia contra el piso. Cualquier fallo anota el motivo y sigue, sin emitir. La sustitución ocurre **después** de comprobar que la lista de rechazos está vacía, así que `values` tiene una entrada por cada ranura de T , todas provenientes de una creencia vigente que pasó el piso; y la sustitución reemplaza ocurrencias del patrón de ranura dejando intacto el resto de la plantilla, que es texto fijo escrito por el código. Las tres condiciones se corresponden una a una con las tres guardas, y no hay un cuarto camino por el que un valor llegue a la salida. ■

Falla cerrada y falla ENTERA, que es una decisión y no una consecuencia. Si una sola ranura no llega al piso, no se emite una versión parcial. Emitir «*el saldo de la cuenta es ___*» no es más honesto que emitir un número inventado: es el mismo acto con mejor caligrafía, e invita al lector a completar lo que el contrato rechazó.

Cuatro límites, dichos adentro del teorema y no en una nota al pie.

límite	qué significa
el alcance es la ranura, no la oración	« <i>el saldo NO supera {x}</i> » con x correcto es sound y falso . No es un defecto de la implementación: es la frontera de la familia entera, y un red-team la mide en vez de suponerla
la procedencia es del registro, no del mundo	<code>COMPUTED</code> significa que alguien la computó y la asentó. El teorema traslada confianza desde el piso hacia la salida; no la crea. Un sensor que miente se emite con procedencia impecable
vigente, no histórica	la garantía es sobre el estado de creencias al ensamblar , no sobre todo lo que alguna vez se creyó
<code>str()</code> es parte del teorema	la condición 3 dice <code>str(valor)</code> , no «el valor». El ensamblador no formatea, porque formatear sería empezar a decidir algo sobre el número

Verificado exhaustivamente y no por casos elegidos: sobre el producto cartesiano de las cuatro procedencias por las condiciones de rechazo — un espacio chico, y por eso uno que se puede recorrer entero.

5.4 La cota nativa del ratchet

El piso de garantía aprendido sólo sube. Un borrador anterior lo acotaba importando un teorema sobre **varianza bajo oscilación**; eso no era una cita floja sino un **error de categoría** — una secuencia monótona y acotada tiene varianza que tiende a cero por construcción, así que la cota se cumplía vacuamente y no decía nada. Lo que un ratchet necesita que se le acote no es cuánto **oscila** sino cuánto **daño acumulado** puede hacer antes de detenerse, y eso es un conteo.

Los niveles son `EXPLORATORY < STANDARD < ACCOUNTABLE < CERTIFIED`, y el techo aprendido es el tercero: `CERTIFIED` queda fuera de alcance a propósito, porque ese nivel restringe qué patrones son admisibles y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad.

Proposición 4 (daño total acotado). Sobre R regiones, el número total de eventos de endurecimiento en **toda la vida del sistema** es $\leq 2R$, sea cual sea la cantidad de ciclos de consolidación.

Demostración. Monotonía: el piso de cada región es una secuencia no decreciente en un conjunto finito y totalmente ordenado, así que cambia a lo sumo tantas veces como niveles haya por encima de su base. No hace falta nada probabilístico. ■

Proposición 5 (la guarda de replicación es fuerte lejos del umbral y débil cerca). Con q la tasa verdadera de rechazo de la región y dos splits de tareas **disjuntos**:

q	un split	ambos	$\approx$
0,10	0,0050	0,000025	1 en 39.613
0,25	0,1138	0,01295	1 en 77
0,40	0,4059	0,16477	1 en 6
0,45	0,5230	0,27358	1 en 4

Eso se dice y no se esconde, e importa menos de lo que parece por dos razones estructurales. Cerca del umbral un falso positivo es casi indistinguible de un verdadero —una región cuyo q real es 0,45 **efectivamente** rechaza casi la mitad de las veces—. Y la Proposición 4 acota el daño acumulado pase lo que pase.

La monotonía que vuelve inaplicable el teorema de varianza prestado es exactamente lo que acota el daño de su propia tasa de falsos positivos. La propiedad que rompe la cota prestada es la que la hace innecesaria.

Y lo que cuesta está medido, lo cual corrige cómo se lee la Proposición 4.

nivel	brazos admisibles	cobertura	u (mejor fijo)
A0 · A1 · A2	5	100%	0,6101
A3	2	40%	0,4221

El ratchet es **gratis hasta A2 y cuesta todo de una vez en A3**: la única transición con precio se lleva **60% del catálogo y 31% de la utilidad**. «A lo sumo dos subidas» invita a imaginar un daño que se acumula despacio; lo medido es lo contrario — **una sola transición tiene precio, y ahí es abrupto**. Las otras dos son gratis porque no hacen nada. Y el promedio esconde a quien paga: una región pierde **−0,5000** mientras la media del corpus es 0,0000.

5.5 Quién fija el dial

La pregunta parece de gobernanza y es de diseño. Si el nivel de garantía lo elige el llamador, un llamador apurado lo baja; si lo elige el sistema, el llamador no puede pedir más rigor del que el sistema cree necesario. **Ninguna de las dos.**

nivel efectivo = max(pedido , piso de creencias , piso aprendido)

fuelle	quién la produce	qué puede hacer
pedido	el llamador, en el request	subir, nunca bajar
piso de creencias	required_floor(Γ) sobre las creencias COMPUTED del request	subir, y no se puede desactivar
piso aprendido	θ.floors[región] , dentro del bundle firmado	subir, y sólo si está promovido

Proposición 6. `max` es la **única** composición bajo la cual cada fuente sólo puede endurecer. Con `min` o con un promedio, agregar una fuente podría ablandar el resultado — y entonces una fuente nueva sería un **riesgo** en vez de una garantía.

Por qué el llamador sube y no baja. Conoce cosas que el sistema no: que este request va a un informe regulatorio, que el resultado se publica, que hay un auditor mirando. Nada de eso está en el material. Lo que no puede es pedir **menos**, porque el piso sale de propiedades **del request mismo** — `irreversible` eleva a A3, `shared_writes` a A2, y las dos entran como creencias `COMPUTED` **declaradas por el caller, nunca inferidas del texto**. Un llamador que pudiera bajar el piso podría declarar una acción irreversible y después pedir tratarla como exploratoria, que es exactamente la combinación que el piso existe para impedir.

Una cuarta fuente, que no es un nivel sino una degradación. A2 admite creencias `ELICITED` , pero sólo una vez que la calibración se ganó; admitirlas antes anula el propósito del nivel. Así que la resolución no baja el nivel: **endurece el piso de procedencia dentro del nivel**. La misma idea del `max` , aplicada al otro eje.

Y se evalúa marginalizando sobre sus posiciones, no fijando una — reportar métricas a un dial fijo reporta una política, no un sistema. Marginalizar produjo un hallazgo sobre el propio dial:

Tres de las cuatro posiciones son indistinguibles. A0, A1 y A2 declaran `admissible_patterns = None` , así que **el dial no restringe el catálogo hasta A3**. Dos de sus tres transiciones no hacen nada en esa dimensión, y toda la diferencia se paga en un solo escalón.

Lo que sí distingue A1 de A2 vive en otros ejes — θ firmada, log de creencias, profundidad de composición, y el piso de procedencia—, así que el dial no es inerte ahí: es inerte **en la dimensión que esa tabla mide**. Decir cuál es cuál es el punto de marginalizar.

5.6 Lo que la teoría no supone, y por qué eso es la afirmación

Todo lo de §5.1–§5.5 está enunciado sobre un catálogo de brazos, una utilidad, una base de creencias y un retículo de procedencia. **Ninguno de los cinco resultados menciona recuperación, documentos ni respuesta a preguntas.** No es un accidente de redacción ni una salvedad: es la afirmación. Lo que se describe es una capa de decisión sobre *acciones que el sistema puede tomar*, y el ruteo de paradigmas sobre un corpus documental es la instancia que pudimos medir sin juez.

La distinción sobre la que el motor realmente gira no es *qué clase de tarea* sino **qué se sabe y cuándo**. Una regla sólo puede gobernar si se la puede evaluar al momento de decidir; un valor sólo se puede emitir si una creencia vigente lo lleva a la procedencia exigida; un nivel sólo se puede subir. Las tres son propiedades de la decisión, no del dominio.

La misma maquinaria, enunciada sobre cuatro superficies. Los teoremas de arriba son la forma general; las columnas son lo que instanciarlos exige.

superficie	el sensor emite	la regla decide	el registro guarda
contenido	números con procedencia	emitir o negarse, por ranura (§5.3)	qué creencia llenó qué ranura
datos	una consulta propuesta, su grano, su resolución temporal	admitir la consulta o exigir elicitación	la consulta, y contra qué se la chequeó
acciones	una llamada a herramienta y sus precondiciones	el piso de procedencia sobre lo irreversible (§5.5)	un ledger de idempotencia
gobierno	una edición candidata de la política	la guarda de promoción sobre episodios held-out	el diff entre dos bundles firmados

La selección entre topologías de control es la **primera** columna instanciada sobre un catálogo de recuperación. Es el caso medido, no el alcance de lo que se afirma.

Y la rama no probada de la teoría y la superficie no medida son el mismo lugar. §5.2 parte el problema sobre v , la disponibilidad de un detector barato, y §1.3 registra que todos los corpus de este registro caen del lado $v = 1$ **por construcción**, porque el gold es lo que vuelve la calificación libre de juez y el gold es un detector. Así que la rama $v = 0$ —la que necesita un router— no la puede alcanzar ningún benchmark que califique por exact-match.

¿Dónde está $v = 0$, entonces? Predominantemente en la superficie de acciones. Chequear *si un archivo se escribió* es barato; chequear *si éste era el reembolso correcto* no lo es, y no hay clave de respuestas que lo abarate. El dominio que este registro no mide es el dominio donde la partición central de la teoría por fin tiene dos lados.

Así que la ambición se enuncia en vez de matizarse. La teoría es general por construcción y está verificada como tal —sobre distribuciones sintéticas con respuesta conocida, no sobre corpus—. Las mediciones son sólo de recuperación, y §8 dice exactamente qué herramientas existieron y cuáles nunca. **§5 se afirma para agentes en general; §7–§8, para extracción de respuesta exacta sobre documentos.** La brecha entre las dos es el alcance real, y está declarada en vez de estrechada.

6. Diseño

6.1 Medir sin juez

Cada tarea trae un oráculo de conjunto, así que la calidad es F1 de conjuntos tras normalización. §2.7 da la razón: el efecto es de un dígito de puntos porcentuales y la varianza del juez es del mismo orden, así que un juez no agregaría solamente ruido sino un **sesgo** — el sesgo de verbosidad favorece las respuestas largas que producen los paradigmas caros, que es precisamente la comparación bajo prueba.

Un oráculo vacío es una pregunta legítima e importante — *listá todos los X* donde no hay ningún X — y testea si un paradigma inventa items. Se califica exigiendo un enunciado explícito de vacuidad; el silencio saca cero, porque un paradigma que no devolvió nada porque se cayó no debe puntuar igual que uno que buscó y reportó no haber encontrado nada.

6.2 Creencias, procedencia y garantía por solicitud

El determinismo pleno no está disponible con un LLM, y perseguirlo excluyendo al modelo de la decisión descarta información que el modelo tiene. La inversión viene de la base declarada en §2.4: el modelo es un **sensor** que emite proposiciones tipadas, una capa simbólica determinista decide sobre la base de creencias, y la garantía toma la única forma que puede tener:

no "el mismo prompt da la misma respuesta" — falso, siempre sino "la misma base de creencias da la misma decisión" — la estabilidad de decisión de esa base, con la base registrada

Lo que esta sección agrega es la **epistemología de la creencia misma**. En la base la procedencia es un campo de trazabilidad; en este trabajo carga el peso de la decisión como jerarquía tipada: **COMPUTED** (una función pura, credencia 1,0) > **OBSERVED** (medido ejecutando una sonda) > **ELICITED** (el modelo lo afirmó, sujeto a calibración) > **ASSUMED**. Una regla puede exigir una procedencia mínima, así que una acción irreversible puede restringirse a evidencia computada y observada: *la opinión de un modelo de que una acción es segura no es evidencia admisible para tomarla*.

La garantía es entonces una propiedad **de la solicitud**, no del sistema. Un modo global hace que todo el tráfico pague por la solicitud más estricta. Cuatro niveles graduán el piso de procedencia, si θ puede aprender online, si la corrida es reproducible desde caché, y **qué patrones son admisibles** — el nivel certificado excluye topologías cuyo flujo de control es no acotado, no porque sean peores (a menudo son mejores) sino porque sus modos de falla no son enumerables. El piso se deriva de creencias sobre la solicitud: quien llama puede pedir más y nunca menos. La derivación misma aprende, offline: una categoría de solicitudes cuyas afirmaciones elicitedas son rechazadas repetidamente por la compuerta es una categoría cuyo piso sube — las estadísticas de rechazo son evidencia sobre la clase de solicitud, y consumirlas cierra el bucle sin ajustar jamás nada adentro de una solicitud.

Tres propiedades lo vuelven seguro de correr, y cada una está impuesta, no pretendida.

El evento es tipado, no parseado. Un rechazo lleva su motivo como valor — la procedencia que se tenía contra la que la regla exigía — de modo que la estadística cuenta el evento. Contarlo emparejando

subcadenas de la explicación mediría el fraseo, y el fraseo es prosa que se reescribe. Sólo cuentan los rechazos por PROCEDENCIA insuficiente: una creencia rechazada por credencia baja, o por tener el valor equivocado, es el sistema funcionando, y no dice nada sobre el régimen de evidencia de la clase.

La guarda es replicación, no utilidad. El registro se parte por tarea; una mitad propone las regiones cuyo piso debería subir, y el piso se instala sólo si la otra mitad —solicitudes que la propuesta nunca vio— dice lo mismo de manera independiente. La utilidad sería el criterio equivocado para esto, y descartarla no es una concesión: subir un piso hace que el sistema exija evidencia medida donde habría actuado sobre una afirmación, lo que cuesta tokens y sólo puede bajar la utilidad medida en el corto plazo. Un piso de gobierno puntuado por la utilidad que produce es un piso que nunca sube.

El aumento es acotado y monótono. Se detiene en accountable y nunca llega a certified, porque certified además restringe qué patrones pueden correr y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad — un piso que aprende no puede quedar habilitado a descalificar una topología. Y nunca baja solo: la ausencia de rechazos después de que un piso sube es justamente lo que ese piso se instaló para producir, así que leer esa ausencia como motivo para bajarlo sería una oscilación puesta en el diseño.

Los pisos aprendidos viajan en el bundle de política firmado, así que nada puede elevar el nivel de una solicitud salvo por el mismo camino de promoción que recorre θ . Verificado: una región cuyos rechazos replican a través de la partición recibe su piso; una región que califica sólo en la mitad que la propuso, no.

Credencia y tamaño de efecto no deben confundirse. Un margen aprendido es una creencia *certera* sobre un efecto *grande* — credencia 1,0, valor 0,9 — y codificar la magnitud como credencia reporta un hecho computado como incierto, destruyendo la distinción para la cual existe el motor.

6.3 Consolidación de la política de control

El aprendizaje es offline y copy-on-write. Los episodios se reproducen en orden de **sorpresa** y no cronológico, lo que elimina el sesgo de recencia que la actualización online tiene por construcción; las estadísticas se reducen en escala y las entradas sin uso se podan; y una etapa de abstracción busca particiones de features que separen paradigmas mejor que el binning actual.

Dos disciplinas hacen esto seguro. Una política candidata se instala sólo si no regresa sobre episodios retenidos. Y el registro se parte en tres **por tarea** — una parte propone particiones, una las puntúa, una la toca sólo el guardia de promoción — porque buscar muchas particiones contra un solo holdout es la forma en que detectar una verdad se vuelve confabularla. Una partición descubierta entra con cero episodios, por debajo del piso de confianza, y no puede gobernar una decisión hasta haber ganado evidencia: **un sueño es una hipótesis, no un hecho.**

Verificado: dado un registro donde la utilidad es independiente de todo atributo, ninguna partición sobrevive la validación.

Cómo se implementa una política así, a modo de ejemplo. Una política de control que es «tabla de condiciones sobre features, versionada y firmada, ejecutada como código» tiene una implementación natural en un motor de *policy-as-code* —Open Policy Agent con reglas en Rego, o equivalente—: las reglas quedan como artefacto de texto revisable, la evaluación es determinista y el motor devuelve permitir/denegar junto con la regla que decidió. **Es un ejemplo de realización en producto y no parte de lo medido:** nada de este paper corre sobre OPA, y la capa de decisión no depende de ningún motor externo (§9.1).

6.3.1 El sistema aprende con los pesos del sensor congelados

El sensor tiene los pesos congelados. No aprende de este despliegue, no guarda nada entre requests, y dos llamadas idénticas no se enteran una de la otra. Y sin embargo **el sistema cambia de comportamiento con la experiencia.** Ésa es la propiedad, y sale de un mecanismo de tres pasos:

1. **Cada decisión deja su huella epistémica.** Al registro va la creencia que la disparó, su procedencia, el paradigma elegido y lo que salió. No es un log de texto: es la base de creencias creciendo, con la misma estructura tipada que gobierna una decisión en vivo.

1. **La consolidación reproduce ese registro offline** —en orden de sorpresa, no cronológico— y emite una **política**: una tabla de condiciones sobre features, versionada y firmada, que se ejecuta como código determinista.

1. **La política entra sólo si no regresa** sobre episodios retenidos, y una partición nueva arranca por debajo del piso de confianza: no puede gobernar hasta haber ganado evidencia.

Así que el aprendizaje vive **en la base de creencias y en la política que se destila de ella**, nunca en pesos. Y de ahí sale lo que hace a esta forma valer la pena, porque son dos cosas que normalmente se pagan una con la otra:

Un sistema plástico suele ser opaco, y uno auditable suele ser fijo. Un artefacto entrenado extremo a extremo aprende y no se puede leer; una tabla de reglas escrita a mano se lee y no aprende. Poner el aprendizaje en la base de creencias en vez de en pesos da las dos: lo aprendido es un artefacto **legible, diffeable, versionado y revertible**, y su instalación pasa por una guarda. Se puede preguntar *qué* aprendió el sistema y contestarlo mostrando dos versiones de una tabla.

La plasticidad es una propiedad del diseño, y hereda su forma de una genealogía que §2.9 hace explícita: consolidar episodios en reglas ejecutables es lo que los sistemas de producción hicieron con *chunking* —Soar, ACT-R—, y que un par de decisiones tenga contenido propio por encima de sus marginales es plasticidad hebbiana llevada a decisiones en vez de a neuronas. Lo que se agrega es sostenerla cuando quien emite las creencias puede inventar con forma correcta.

6.3.2 Sobre qué superficies

Lo que el motor aprende **no es un modelo**: son **políticas**, que consolidan offline en un artefacto firmado y versionado y después se ejecutan como código determinista.

superficie	qué se aprende
ruteo	qué paradigma admite una región de features
herramientas	qué índice sirve a una consulta según su clase (§7.2.4)
pares de herramientas	qué asociación <code>tool_i → tool_j</code> predice, que un histograma marginal no captura
handoff	cómo repartir el alcance entre sub-agentes
comportamiento	el piso de garantía sube donde el registro muestra que las aserciones del modelo se rechazan
credencia	calibración por proposición en vez de un interruptor global de confianza

Las dos últimas son las que cambian lo que el sistema *promete*, no lo que elige: un modelo puede estar bien calibrado sobre cardinalidad y ser inútil sobre acoplamiento, y una política de confianza global no puede representar eso.

6.3.3 De dónde viene esta forma

El linaje está en §2.9, con la tabla de qué supone cada línea sobre su fuente de creencias y qué de eso rompe un sensor estocástico. Lo que importa para el argumento es la consecuencia de diseño: **la procedencia no se deriva, se declara y se verifica**, y esa inversión es la que permite que las ocho líneas sigan funcionando cuando la fuente puede inventar con forma correcta.

6.3.4 La afirmación de transferencia, y su refutación medida

La lectura se completa con un condicional: *si el corpus es representativo del dominio, la capa ajustada debería transferir*. Eso es falsable, y este registro ya lo falsó una vez.

P15. Sobre un mundo que la política nunca había visto —seed 47, 390 celdas, cero errores de infraestructura— el ruteo por request perdió contra el mejor paradigma fijo por **−0,087**, más allá del piso de ruido, **mientras reproducía cada decisión 26/26** desde su base de creencias registrada.

El mecanismo es el hallazgo, no el número. El vocabulario de región **no tiene eje de horizonte**, así que las tareas que castigan una elección fija eran indistinguibles de las que la premian. La política ruteó contra su propio veredicto registrado porque **ninguna etiqueta le dijo nunca en qué caso estaba**. Chequeo de sensibilidad: reparar la validez del aprendizaje —agregación por episodio, holdout limpio— deja el número idéntico, así que la refutación no es un artefacto del procedimiento.

Eso afila la condición hasta volverla chequeable:

La representatividad hay que enunciarla sobre los ejes que el vocabulario de región distingue. «Representativo del dominio» no alcanza. Un corpus que varía en una dimensión que el mapa de features no mira produce episodios que la política no puede separar —y entonces aprende un promedio sobre dos poblaciones. La condición se puede chequear sobre un corpus *antes* de correrlo, porque la región es una función determinista de los features.

6.3.5 Una consecuencia afirmable

Más inferencia no compra más ajuste. La consolidación es replay sobre el registro, así que el costo de aprender es **cero llamadas**: lo que la cuota compra son *episodios*, y el ajuste es gratis sobre los que haya. Eso separa dos decisiones que se suelen tomar juntas — cuánto medir lo gobierna la potencia estadística, cuánto entrenar no lo gobierna nada.

Y es observable mientras una campaña corre. Sobre la campaña actual, **516 filas dan 141 episodios sobre 6 regiones, 57 pares** (región, paradigma) **con evidencia y 27 con $n \geq 3$** — leído del registro entre dos tandas, sin costo adicional. La potencia estadística la fija el **corpus**, no la cuota: un par junta un episodio *por tarea* de su región, así que cruzar el piso de evidencia exige esa cantidad de tareas. Gastar más agrega tareas, y sólo cuentan si caen en la región justa.

7. Resultados

7.0 Preguntas de investigación

Las mediciones de esta sección contestan cinco preguntas, y cada una tiene un criterio de decisión fijado **antes** de mirar el dato. Las predicciones falsables quedaron registradas con fecha en el cuaderno del laboratorio; en esta tabla se enuncia lo que cada una decide.

	pregunta	criterio	sección	
PI1	¿Cuánta varianza de trayectoria hay en el régimen no confinado, y confinarla cambia el resultado?	fracción de celdas con <code>pass^k < pass@1</code> , y el efecto de absorber ramificaciones sobre una celda con <code>d > 0</code>	§7.2	
PI2	¿Elegir paradigma por tarea captura brecha de oráculo neta positiva?	brecha observada menos el piso de ruido por remuestreo, en muestra y sobre held-out	§7.3, §7.7	
PI3	¿Existe una señal disponible al decidir que explique la interacción tarea×paradigma?	fracción de <code>γ</code> explicada, contra el máximo de los nulos por permutación (corrección por selección)	§7.3.2	
PI4	¿El acuerdo entre paradigmas predice corrección sin oráculo ni juez?	<code>^P(correcta \</code>	k coinciden)`, con tres controles: dificultad de tarea, largo de respuesta y costo	§7.5
PI5	¿Las capacidades declaradas transfieren a un paradigma no visto?	error de predicción dejando un paradigma afuera, contra el nulo de capacidades barajadas	§7.4	

Tres términos que el paper usa con precisión. Una **celda** es un par tarea × paradigma, y es la unidad de medición. Un **modo** es una de las once clases de falla del corpus (§1.3). La **etiqueta de diseño** es el modo del que salió una tarea: se conoce al construirla y no al decidir, así que en §7.3.2 funciona como cota superior y nunca como señal disponible.

Régimen de medición. Doce paradigmas sobre condiciones idénticas —mismo corpus, mismo modelo, misma superficie de herramientas, misma decodificación— con `repeat = 3`, sin juez LLM, con el corrector auditado (§7.0.1), y cada panel declarando su rectángulo. Los fallos de infraestructura se registran como tales y se excluyen de toda estadística: la campaña tuvo cero.

7.0.1 Validez del corrector

El pitfall más caro de un banco de agentes es que sus ceros no sean del sistema sino del corrector: una auditoría manual de fallos en un benchmark público de código encontró que el **59,4%** de los fallos atribuidos al modelo eran defectos del arnés de evaluación. Acá el corrector se audita con una prueba automática que barre toda respuesta con utilidad cero y marca las que traen señal de defecto de

emparejamiento —el oráculo presente en la respuesta sin sobrantes, un rechazo no reconocido, un ítem parcial no acreditado, un número equivalente con otro formato—. Sobre **567** ceros, **ninguno** presenta señal fuerte. La auditoría no prueba que el corrector sea correcto: prueba que esas cuatro clases de defecto no están.

La campaña homogénea, y es la única fuente de los números de esta sección. Doce paradigmas × 78 tareas × 3 réplicas sobre un corpus con entidades reales, un modelo, mismas condiciones, sin juez LLM y con el corrector auditado: 121,4M tokens, cero errores de infraestructura. Toda u usa $\lambda = 0$ —calidad pura, con el costo en su propia columna— y cada panel declara su rectángulo.

7.1 Los doce paradigmas, en una tabla

Las secciones que siguen miden a cada brazo por un lado distinto —cobertura, degradación con el ancho, fiabilidad, latencia, decisiones delegadas— y cada una tiene su tabla. Ésta las junta, porque **cinco tablas que nadie cruza son menos útiles que una que declara sus denominadores.**

brazo	aplica	u	u × aplica	pass^3	tok/celda	USD/1k celdas	serie	ley de costo
react	100%	0,850	0,850	0,734	108.137	22	1,35 s	vueltas
dag_strategy	100%	0,830	0,830	0,703	105.293	22	2,87 s	vueltas
reflection	100%	0,808	0,808	0,688	133.574	27	1,76 s	vueltas
rewoo	100%	0,678	0,678	0,531	10.840	2	0,77 s	estructural
supervisor	100%	0,591	0,591	0,406	64.079	13	2,66 s	vueltas
gist_reader	100%	0,584	0,584	0,516	23.075	5	0,91 s	vueltas
handoff	96%	0,583	0,557	0,438	131.310	27	2,04 s	alcance
pointer_chase	96%	0,515	0,492	0,359	9.787	2	1,38 s	vueltas
graph_traverse	56%	0,511	0,288	—	21.356	4	—	estructural
streaming_scan	12%	0,750	0,090	—	26.380	5	—	estructural
extract_compute	12%	0,583	0,070	—	26.184	5	—	estructural
direct	6%	0,917	0,055	—	22.822	5	—	estructural

Los denominadores, y son dos. `aplica`, `u`, `u × aplica`, `tok/celda` y `USD` salen del registro entero: `aplica` es qué fracción de las celdas **ofrecidas** a ese brazo pasa el compuerta aritmética de factibilidad, y `u` promedia sólo las que pasan. `pass^3` y `serie` salen del rectángulo de **64 tareas × 8 brazos** —el 82% de las medidas— porque exigen que todos los brazos hayan corrido las mismas tareas con tres réplicas cada una; los cuatro brazos sin valor son los que la factibilidad poda en casi todas las celdas. Toda u usa $\lambda = 0$: calidad pura, con el costo en su propia columna.

Cuatro cosas que sólo se ven con las columnas juntas:

1. `u` y `u × aplica` son dos números y ninguno reemplaza al otro. `direct` es el mejor

del plantel donde su mecanismo corre — 0,917 — y aporta 0,055 sobre el corpus porque corre en el 6% de las celdas. No falla en el 94% restante: **no corre**, y eso lo decide la aritmética antes del primer token.

1. **pass³ siempre está por debajo de u, y la distancia no es proporcional.** react

pierde 0,116 y supervisor 0,185. Esa diferencia es varianza que vive *dentro* de una celda, invisible para cualquier piso de ruido calculado entre brazos.

1. **La columna de dólares no es la de tokens reescalada.** Entrada y salida se cobran 6×

distinto, y los brazos se diferencian justo en esa proporción — handoff y reflection cuestan lo mismo en dólares con 2.264 tokens de diferencia por celda.

1. **El margen está en la última columna, no en la primera.** Entre react y rewoo hay

0,172 de utilidad y un factor 11× de costo y 1,8× de latencia serial. En utilidad los brazos se separan por centésimas; en lo que cuestan, por órdenes de magnitud.

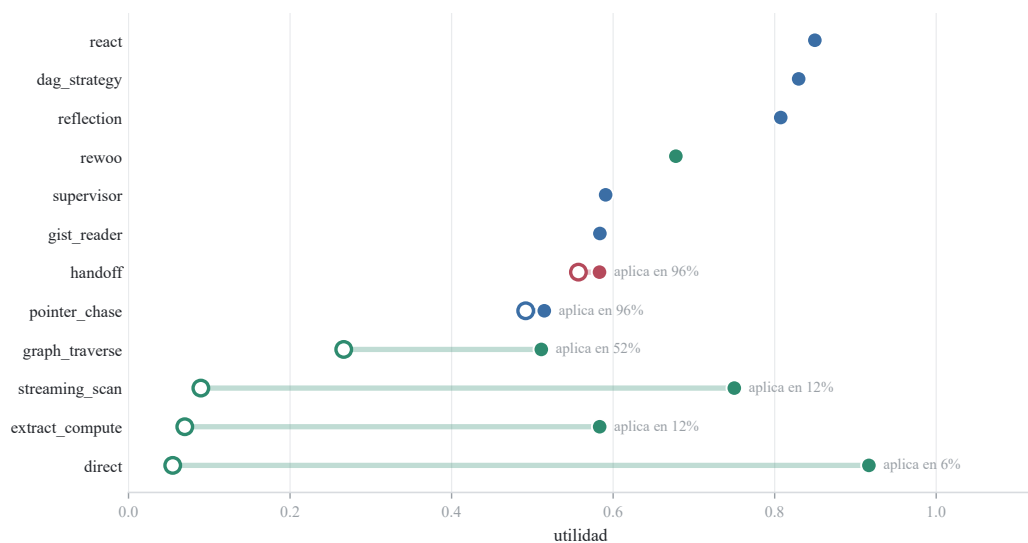
El pass³ de esta tabla y el de §7.2.2 son rectángulos distintos y no se deben cruzar. Acá sale de las 64 tareas comunes a los ocho brazos; allá, del subconjunto donde los cuatro brazos comparados corrieron sus tres réplicas, que es más chico y más fácil. De ahí que react figure con 0,734 en esta tabla y 0,797 en §7.2.2.

pointer_chase **figura en esta tabla con su número de campaña.** Las cuatro correcciones de §7.2.4 —que lo llevan de 0,33 a 0,89 en la celda de cadenas acopladas— son posteriores a esta corrida y tocan 3 de las 78 tareas, así que su efecto sobre el agregado del corpus está dentro del ruido y **no se propagó a esta tabla.** Se dice en el cuerpo y no al pie porque una tabla que mezcla dos versiones del mismo brazo sin declararlo es exactamente el defecto que este paper audita en otras partes.

7.1.1 Un paradigma tiene dos números y colapsarlos esconde el caso que importa

brazo	aplica	u donde aplica	u × cobertura	tokens/celda
react	100%	0,850	0,850	108.137
dag_strategy	100%	0,830	0,830	105.293
reflection	100%	0,803	0,803	133.574
rewoo	100%	0,669	0,669	10.840
gist_reader	100%	0,584	0,584	23.075
supervisor	100%	0,581	0,581	64.079
handoff	96%	0,583	0,557	131.310
pointer_chase	96%	0,515	0,492	9.787
graph_traverse	52%	0,511	0,266	21.356
streaming_scan	12%	0,750	0,090	26.380
extract_compute	12%	0,583	0,070	26.184
direct	6%	0,917	0,055	22.822

Un paradigma tiene dos números, y colapsarlos esconde el caso extremo
relleno: utilidad donde su mecanismo corre · hueco: multiplicada por su cobertura



Bueno donde aplica, contra lo que aporta sobre el corpus

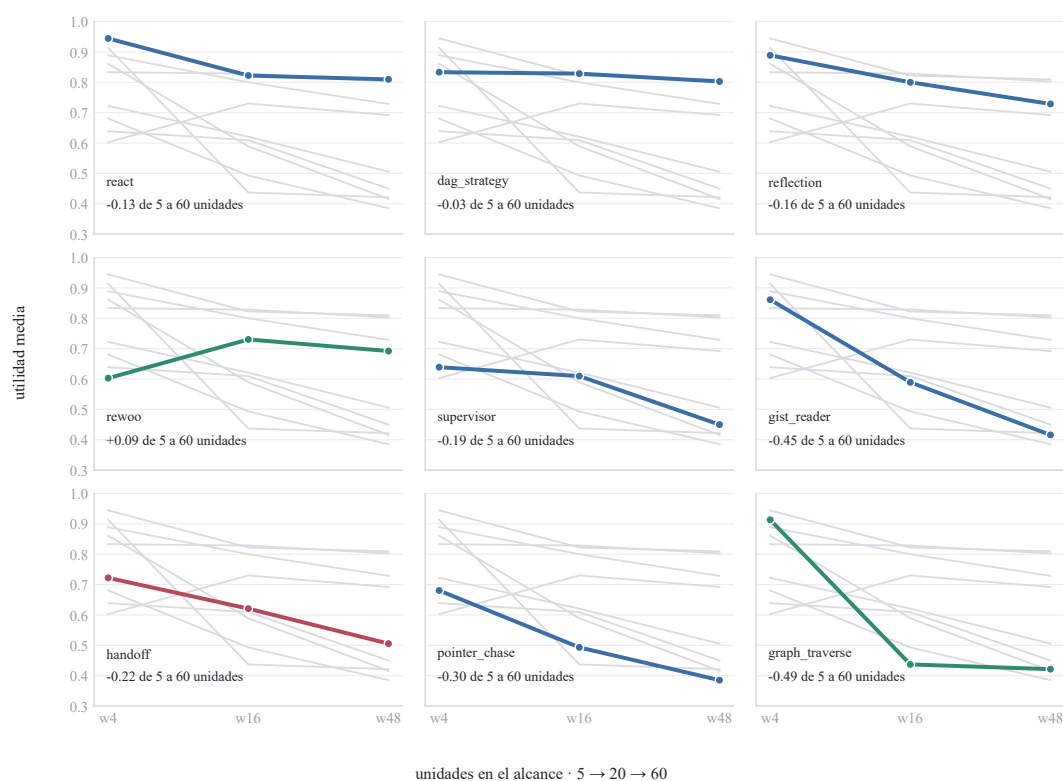
Un paradigma tiene dos números y colapsarlos esconde el caso que importa. `direct` es el mejor del plantel donde su mecanismo corre —0,917— y su mecanismo corre en el **6%** de las celdas —**12 filas de 201**, y ese número va junto al porcentaje porque «el mejor del plantel» sobre doce filas es una afirmación de otra clase que sobre doscientas—, porque la aritmética de factibilidad lo poda en cuanto el material no entra. Sobre el corpus aporta 0,055. Reportar un solo número obliga a elegir cuál de las dos afirmaciones falsear.

La cobertura no es una propiedad del brazo sino de la **intersección** entre su mecanismo y la distribución de tareas, y por eso se decide antes de gastar: `direct`, `streaming_scan` y `extract_compute` no fallan en el 88–94% restante — **no corren**.

7.1.2 La degradación con el ancho separa lo que la utilidad media junta

Casi todos se degradan cuando el material crece. `rewoo` no.

un panel por brazo; los demás, en gris de fondo



Cómo se degrada cada brazo cuando el material crece

El eje son los tres anchos declarados —**5, 20 y 60 unidades**—. Las tareas sin sufijo de ancho quedan **fuera**: agrupan celdas de 1, 8, 9 y 60 unidades, así que no son el extremo angosto de nada y meterlas convertía el eje en algo que no está ordenado.

brazo	w4 (5 u.)	w16 (20 u.)	w48 (60 u.)	Δ
react	0,94	0,82	0,81	-0,13
dag_strategy	0,83	0,83	0,80	-0,03
reflection	0,89	0,80	0,71	-0,18
rewoo	0,60	0,73	0,66	+0,06
handoff	0,72	0,62	0,51	-0,22
supervisor	0,64	0,59	0,43	-0,20
gist_reader	0,86	0,59	0,42	-0,45
pointer_chase	0,68	0,49	0,39	-0,30
graph_traverse	0,91	0,44	0,42	-0,49

Todos se degradan menos uno. `graph_traverse` pierde 0,49 y `gist_reader` 0,45 al pasar de 5 a 60 unidades — el gist de 180 caracteres y el índice de entidades dejan de discriminar cuando hay sesenta

candidatos. `rewoo` es la única excepción, y sube: `+0,06` . No es casualidad — es el único brazo cuyo costo no es función del alcance.

Un promedio sobre anchos habría llamado «mejor» a `gist_reader` que a `rewoo` por 0,584 contra 0,669, y habría escondido que uno se desploma exactamente donde el otro se sostiene.

7.1.3 Tres clases de costo, y no son las del catálogo

Ajustando `log(costo)` contra `log(alcance)` por brazo, y preguntando por separado qué explica mejor el costo —el alcance o la cantidad de vueltas— aparece una taxonomía que **corta transversal** a la de control de flujo:

clase	brazos	qué la define
alcance	<code>handoff</code>	$R^2 = 0,77$ contra el alcance, exponente 0,73. El costo lo fija cuánto material arrastra cada llamada
vueltas	<code>react</code> , <code>reflection</code> , <code>dag_strategy</code> , <code>supervisor</code> , <code>gist_reader</code> , <code>pointer_chase</code>	el costo lo fija cuántas veces itera, y eso es endógeno : gasta hasta que algo lo detiene
estructural	<code>rewoo</code> , <code>direct</code> , <code>graph_traverse</code> , <code>extract_compute</code> , <code>streaming_scan</code>	ni una ni la otra: el costo está fijado por la forma del patrón

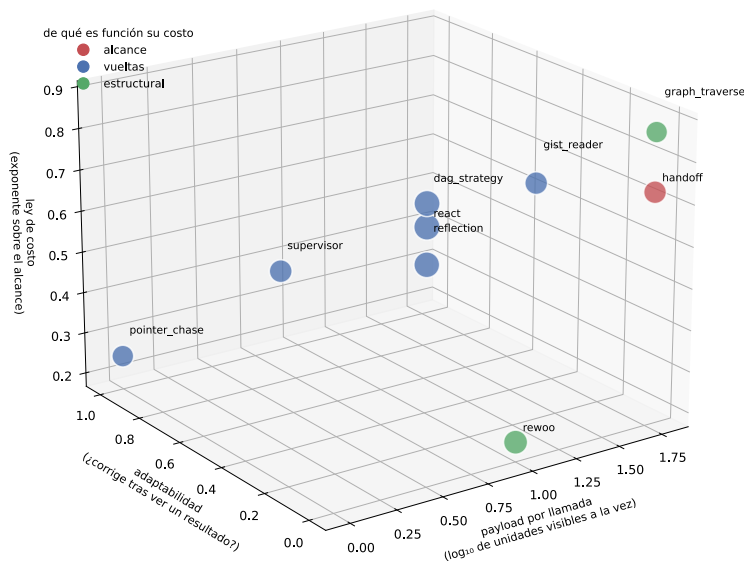
El dato que obliga a separar estas clases de los techos declarados: `handoff` tiene un techo de 12 llamadas y gasta 131.310 tokens; `dag_strategy` tiene uno de 160 y gasta 105.293. Casi lo mismo, con un factor 13 de diferencia en el techo.

Contar llamadas para acotar esfuerzo es contar envases para acotar peso.

Y una consecuencia operativa: la clase **vueltas** es la única sobre la que una regla de parada puede actuar. Medido en el mismo registro, el 46,5% de las búsquedas de `react` no traen ninguna unidad nueva, con rachas de hasta 14.

7.1.4 El espacio de capacidades

El espacio de capacidades: cada brazo, un punto
el tamaño es su utilidad; el color, de qué es función su costo



El espacio de capacidades

La taxonomía de control de flujo —«plan-ejecuta», «supervisor», «cadena»— no predice rendimiento. Lo que sí predice es qué **capacidades** le da cada topología al modelo, y son tres:

eje	qué es	de dónde sale
payload por llamada	cuántas unidades ve el modelo de una vez	del código
adaptabilidad	¿puede corregir el plan tras ver un resultado?	del código
ley de costo	de qué es función su costo	medida

Las dos primeras se leen del código, y por eso **ubican a un brazo que todavía no se corrió** — cosa que una tabla de resultados no puede hacer.

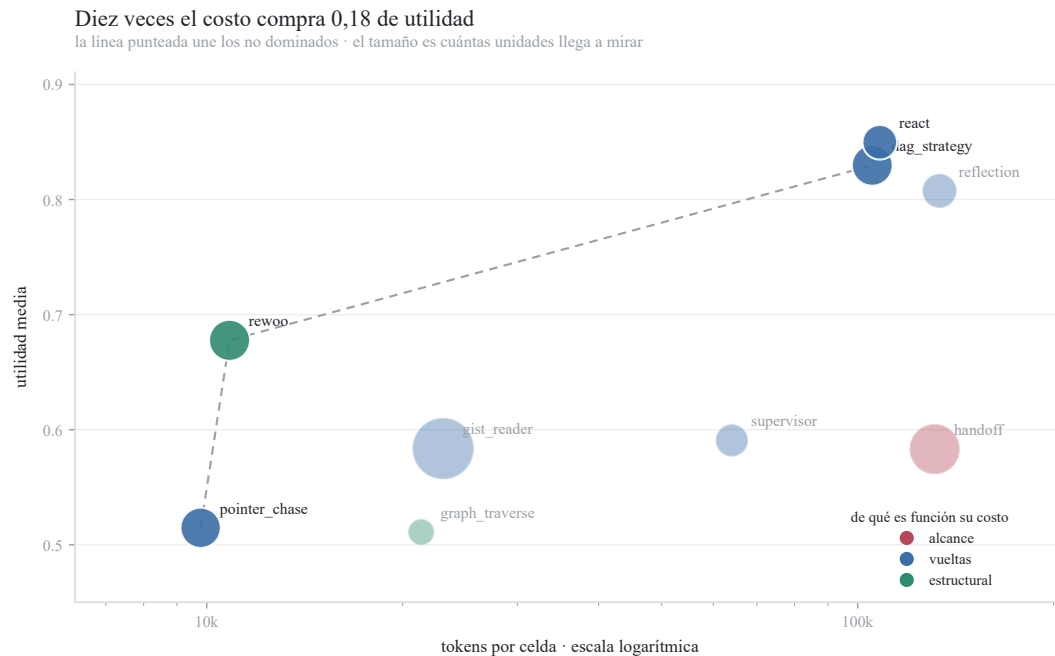
El caso que las valida es la pregunta de contradicción, donde la respuesta es una relación entre dos unidades y ninguna unidad la contiene:

capacidades	utilidad
payload $\geq$ 2 unidades y adaptabilidad	0,71 – 0,91
sólo payload	0,13
ninguna	0,00 – 0,40

handoff lee las dos unidades relevantes y saca **0,067**, porque cada sub-agente ve su mitad y ninguna llamada tiene el par: la capacidad no es «leerlas» sino «tenerlas juntas». Y **rewoo** saca 0,133 aunque *puede* tenerlas juntas, porque le falta la otra — poder elegir **cuáles** dos exige ver un resultado antes de pedir el siguiente.

En la figura, `react`, `dag_strategy` y `reflection` caen prácticamente en el mismo punto. Eso no es un defecto del dibujo: **son el mismo brazo para decidir**, y por eso sus utilidades quedan dentro de 0,05 entre sí.

7.1.5 Dónde está el margen

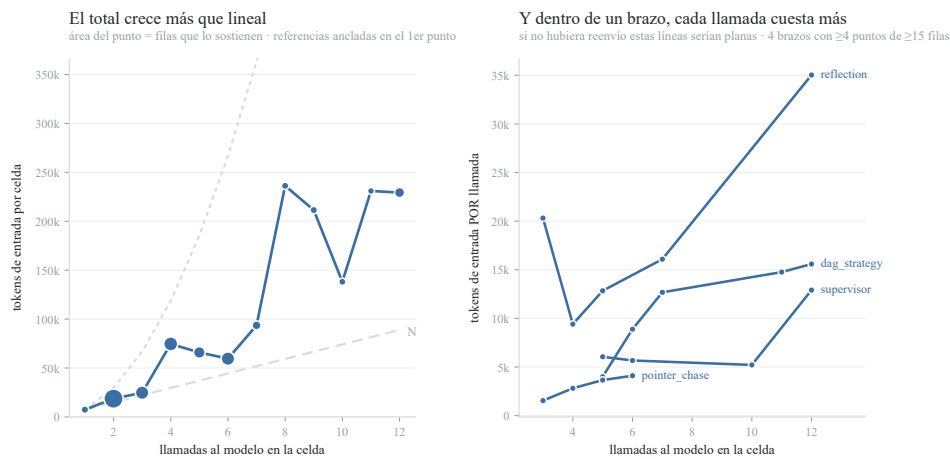


El negocio de cada brazo

En el ancho mayor, `react` saca 0,81 a 108.137 tokens por celda y `rewoo` 0,66 a 10.840: **+0,15 de utilidad por un factor 10 de costo**. Y el costo es casi enteramente de entrada —de **98,6% a 100,0%** según el brazo, con la salida entre 0,0% y 1,4%—, lo cual dice que **los paradigmas no se diferencian en lo que generan sino en lo que arrastran al prompt**. Es la misma afirmación que la ley de costo, medida por otro lado.

Un brazo queda afuera del rango. En `graph_traverse` la suma de entrada y salida da 101,9% del costo registrado. Entrada y salida son las dos particiones del mismo total, así que por encima de 100% no hay una tercera categoría: **es un defecto de contabilidad de ese brazo**, no una propiedad medida. Se excluye del rango y queda anotado como deuda, no como hallazgo.

7.1.6 La ley de costo, dibujada



La ley de costo

El paper afirma desde el principio que el costo de un bucle de herramientas crece como N^2 y la cobertura como N , porque **la conversación se reenvía entera en cada vuelta**. Hasta esto lo sostenían dos números sueltos — ≤ 2 llamadas dan 9.779 tokens, ≥ 8 dan 136.432— y esos dos son compatibles con crecimiento lineal si uno no mira el resto.

Hacen falta dos paneles y no uno, porque un total creciente no distingue «cada llamada cuesta lo mismo y hay más llamadas» de «cada llamada cuesta más». El panel derecho separa las dos: si no hubiera reenvío, **esas líneas serían planas**.

brazo	3-5 llamadas	11-12 llamadas	factor
dag_strategy	4.002	15.592	3,9×
supervisor	6.052	12.910	2,1×
pointer_chase	1.544	4.107	2,7×
reflection	9.410	35.030	3,7×

El panel derecho condiciona por brazo, y esa es la única forma de leerlo. Agregado sobre todos, el costo por llamada zigzaguea —7.411, 18.654, 29.528, 13.815, 19.113— porque distintos brazos dominan distintos conteos de llamadas y sus alcances difieren en un orden de magnitud: «más llamadas» y «qué brazo» quedan confundidos, y el zigzag es esa confusión y no el fenómeno. Condicionado por brazo el trazo sube monótono en los cuatro que tienen puntos suficientes.

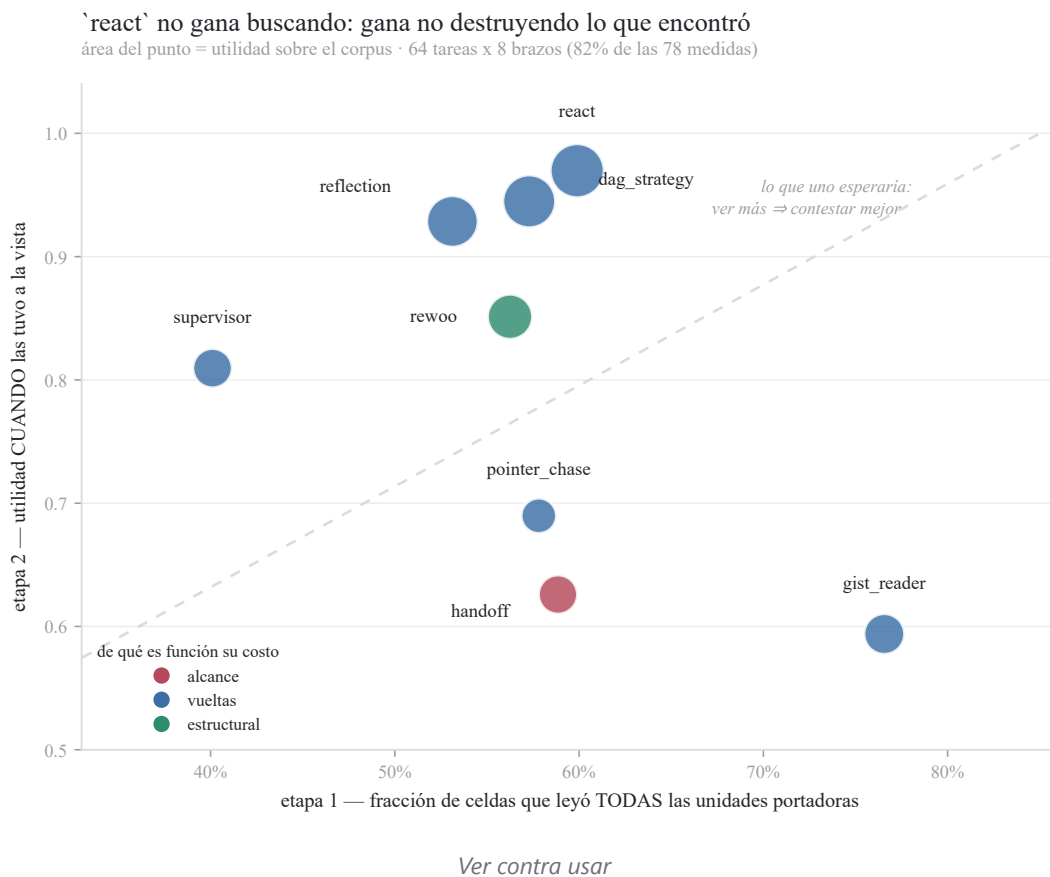
No se estima ningún exponente ni se reporta un R^2 : las curvas N y N^2 del panel izquierdo están ancladas en el primer punto para que el ojo compare, y el hallazgo es cualitativo. Con n despasejo por punto —de 26 a 540 filas— un exponente ajustado tendría más precisión aparente que evidencia.

7.2 Dónde entra la varianza del sensor, y cómo el motor la confina

Las secciones anteriores comparan brazos. Ésta abre uno: **por qué gana el más simple, qué varianza esconde el promedio, y qué pasa cuando una decisión de control se le saca al modelo**. Las tres preguntas se contestan sobre el mismo registro, sin gastar un token más.

CADA PANEL DECLARA SU RECTÁNGULO, y por eso u toma valores distintos según qué se compare. react figura con 0,850 en §7.1.1, 0,843 en §7.2.1 y 0,875 en §7.2.2: son tres poblaciones distintas, cada una elegida por lo que la pregunta de esa sección exige. | sección | panel | por qué ése | |---|---|---| | §7.1.1 | 78 tareas × 12 brazos, cada brazo sobre las celdas donde CORRIÓ | mide cobertura y aporte, que exigen incluir a los brazos podados | | §7.2.1 | 64 × 8 (rectángulo completo) | el embudo compara etapas entre brazos: exige que todos hayan corrido las mismas tareas | | §7.2.2 y §7.3 | 59 × 8 (rectángulo con ≥ 3 réplicas) | $pass^3$ y la descomposición necesitan las tres réplicas de cada celda | **La regla del rectángulo se aplica desde un solo lugar** y devuelve también lo descartado: un panel que se achica sin declarar cuánto miente por omisión. Toda u reportada usa $\lambda = 0$ —calidad pura, con el costo en su propia columna— salvo donde se indique lo contrario.

7.2.1 El embudo: react no gana buscando



La utilidad de una celda es el producto de dos cosas que fallan por razones distintas:

$$u \approx P(\text{vio TODAS las unidades portadoras}) \times P(\text{contestó bien} \mid \text{las vio})$$

brazo	u	vio	**u \	vio**	u \	no vio	leídas
react	0,843	60%	0,970	0,708	7,8		
dag_strategy	0,822	57%	0,945	0,720	11,6		
gist_reader	0,611	77%	0,594	0,667	28,8		
supervisor	0,577	40%	0,778	0,479	6,8		

react no ve más que nadie — 60%, y **gist_reader** ve el 77%. Toda su ventaja está en la segunda etapa: con el material a la vista da 0,970, y **gist_reader** da 0,594, *peor que cuando no lo vio todo*. Pareado por tarea, Δ_{vio} es chico o negativo y $\Delta(u|vio)$ se lleva la brecha entera.

El mecanismo: cada representación intermedia más chica que el material es una pérdida que no se recupera aguas abajo. Un gist de 180 caracteres, una ventana recortada para el sub-agente, un índice de entidades — los tres tiran información *antes* de saber cuál hacía falta. **react** no tiene ninguna.

La simplicidad no es en este caso una virtud estética: es la **ausencia de un canal con pérdida**, y eso se mide.

7.2.2 **pass^k** : el promedio esconde la mitad que importa

pass^k NO es pass@k, y significa casi lo opuesto. En la literatura de código **pass@k** mide «al menos un acierto en k intentos» y **crece** con k . **pass^k** mide «los k intentos acertaron todos» y **decrece** con k . El nombre viene de **tau2-bench**, que lo introdujo para agentes multi-turno por esta misma razón; lo conservamos por consistencia con esa literatura y se señala porque el parecido tipográfico invita a leer la tabla al revés.

Todo lo anterior es **pass@1** —el promedio sobre réplicas— y responde «cuánto acierta». **pass^k** responde «**se puede contar con que acierte**», que para un sistema que promete «misma base de creencias $\Rightarrow$ misma decisión» es la mitad que importa.

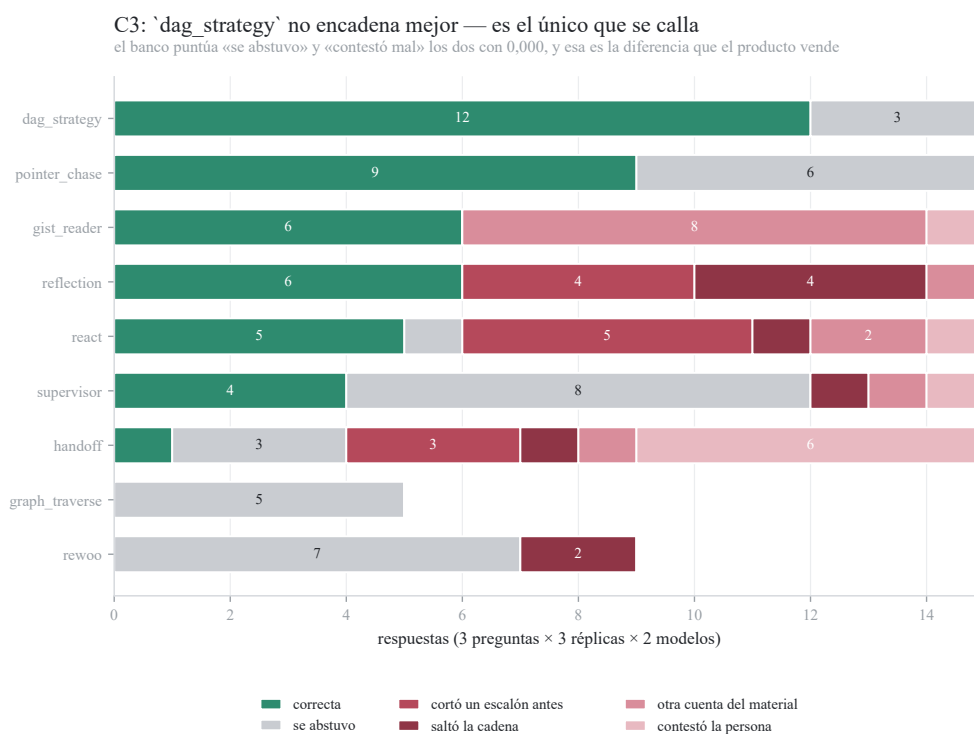
brazo	pass@1	pass³	caída	celdas inestables
react	0,875	0,797	−0,078	17%
dag_strategy	0,874	0,763	−0,112	20%
rewoo	0,718	0,576	−0,142	27%
supervisor	0,637	0,441	−0,196	34%

Entre el 17% y el 34% de las celdas cambian de resultado entre réplicas, con $t=0$, semilla fija y la misma huella.

La varianza de una llamada y la de una trayectoria son cantidades distintas. Sobre una llamada, la varianza del sensor se manifiesta como un token distinto: el resultado se mueve poco y de forma acotada. Sobre una trayectoria, una elección distinta en el paso uno cambia **qué documento se lee en el paso dos**, y de ahí en adelante las dos réplicas ya no comparan la misma evidencia. La varianza no se promedia: se **ramifica**. Si la trayectoria delega d puntos de ramificación a la salida del modelo, la trayectoria es una variable aleatoria sobre un árbol de profundidad d . Con $d = 0$ la trayectoria es **fija**, y la varianza del sensor entra únicamente como el contenido que extrae de cada nodo: un error acotado y localizable en vez de una historia distinta.

Esta varianza es invisible para el instrumento habitual: **vive dentro de una celda**, así que ningún piso de ruido calculado *entre* brazos la muestra. Por eso hace falta `pass^k`, y por eso no basta con más réplicas del mismo promedio.

7.2.3 En una cadena acoplada, el modo dominante es cortarla un escalón antes



Modos de falla de C3

`C3_coupled_chain` pide subir N escalones de una línea de reporte sobre 60 unidades y reportar un dato del último. La cadena está minada a propósito: **cada unidad del camino lleva un dato del mismo tipo pegado al nombre que la ancla**, el enlace va por anáfora («The above-named», «That person»), y el destino del salto viene abreviado (A. Vallejos apunta a Agustina Vallejos).

Clasificar las respuestas por modo, y no por puntaje, invierte el diagnóstico. El modo dominante no es saltarse la cadena (2 casos) sino **cortarla un escalón antes** (7): los brazos devuelven el dato de un intermedio, que está a la vista y es indistinguible del correcto. Y `dag_strategy`, que gana la celda, **no encadena mejor**: hace 12 correctas, 3 abstenciones y **cero respuestas equivocadas**. Los demás contestan igual.

El banco puntúa «se abstuvo» y «contestó mal» los dos con 0,000. Es correcto para medir utilidad y ciego justo en el eje que el motor de decisión existe para gobernar.

7.2.4 Sustituir una ramificación delegada al modelo por un medidor de entorno

Sobre ese diagnóstico se corrigió `pointer_chase` —el brazo cuyo mecanismo es seguir cadenas y que sacaba 0,33 en C3—. Cuatro correcciones, **todas de flujo de control o de tipado, ninguna de fraseo**:

1. ****Regla de creencias**: una entidad nombrada se busca con el índice léxico, no con el

híbrido.** Un vector denso codifica *de qué habla* un texto, y sesenta documentos con la misma plantilla hablan de lo mismo; el nombre propio es justo la parte que **no** es semántica, y fusionarle la rama densa le mete ruido a la única señal que discrimina. Medido: el híbrido devuelve la unidad equivocada para `Ramiro Herrera` y deja a `M. Arrieta` fuera del top-5; el léxico las pone primera y tercera.

1. **La salida del sensor se tipa antes de usarse**. El modelo emitía

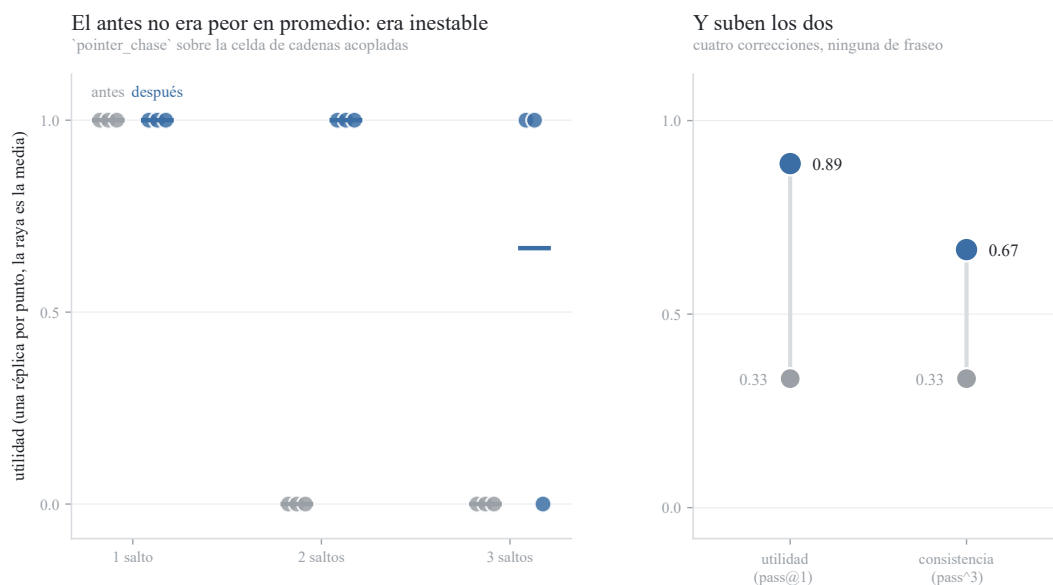
`'M. Arrieta settlement account'`, y esa cola arrastraba la consulta a clasificar como prosa: **la regla era correcta y la entrada estaba sucia**.

1. **Un salto a una unidad que no nombra a quien se persigue no es un salto**, y entre

candidatos empatados gana el que la nombra **antes** — un documento que trata *sobre* una entidad la nombra antes que uno que la referencia de paso. Se resuelve con predicados que devuelven un booleano o una posición, nunca texto: no cuestan un token.

1. **El ancla es el salto cero y lo resuelve el código**. Era una llamada al modelo, y ahí

quedaba la última decisión de flujo en manos del sensor.



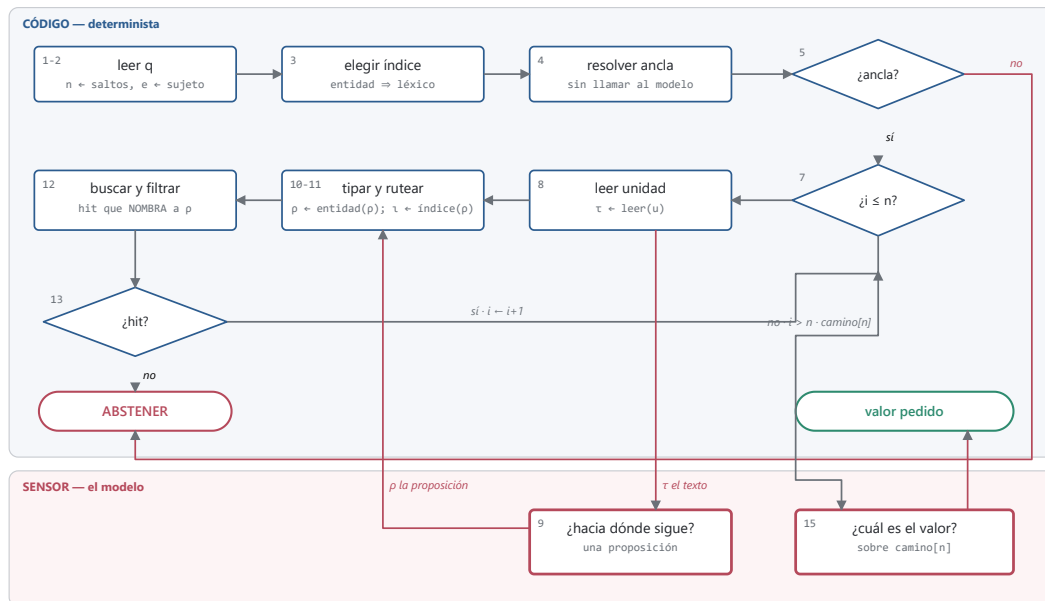
Una ramificación delegada, resuelta por código

El panel izquierdo dibuja **cada réplica por separado**, y ahí se ve lo que un promedio esconde: el «antes» no era peor en promedio sino **inestable** — la misma pregunta, la misma huella y los mismos resultados

de búsqueda daban 1,000 o 0,000 según la réplica. El panel derecho muestra los dos ejes moviéndose juntos, que es la afirmación entera.

Algoritmo 1 — dos de quince líneas llaman al modelo

el carril inferior es el sensor: emite proposiciones y nunca decide el flujo



Flujo de control del Algoritmo 1

Algoritmo 1. La caminata, con la frontera modelo/código explícita. **SENSOR** es la única llamada al modelo y devuelve **una proposición**; todo lo demás lo decide el código.

ALGORITMO 1 Caminata determinista sobre una cadena de referencias

Entrada : pregunta q , vista del corpus V , plantel de índices I

Salida : valor pedido, o ABSTENER

```
1   $n \leftarrow \text{saltos\_declarados}(q)$            ▷ forma cerrada sobre  $q$ ;  $\emptyset$  si no declara
2   $e \leftarrow \text{sujeto\_declarado}(q)$          ▷ forma cerrada sobre  $q$ 
3   $\iota \leftarrow \text{índice\_para}(e)$            ▷ REGLA DE CREENCIAS: entidad  $\Rightarrow$  léxico
4   $u \leftarrow \text{primer\_hit\_que\_nombra}(\text{buscar}(\iota, e, \kappa), e, \emptyset)$    ▷ ancla = salto cero
5  if  $u = \perp$  then return ABSTENER
6   $\text{camino} \leftarrow \langle u \rangle$ 
7  for  $i \leftarrow 1$  to  $n$  do                 ▷ el LARGO lo fija  $q$ , no el modelo
8     $\tau \leftarrow \text{leer}(u)$ 
9     $p \leftarrow \text{SENSOR}(\tau, \text{«¿hacia dónde sigue el rastro?»})$    ▷  $\leftarrow$  única llamada
10    $p \leftarrow \text{entidad\_en}(p)$            ▷ se TIPa la salida del sensor
11    $\iota \leftarrow \text{índice\_para}(p)$ 
12    $u \leftarrow \text{primer\_hit\_que\_nombra}(\text{buscar}(\iota, p, \kappa), p, \text{camino})$ 
13   if  $u = \perp$  then return ABSTENER       ▷ cadena incompleta  $\Rightarrow$  no se responde
14    $\text{camino} \leftarrow \text{camino} \frown \langle u \rangle$ 
15  return  $\text{SENSOR}(\text{leer}(\text{camino}[n]), \text{«el valor pedido»})$    ▷  $\leftarrow$  el CÓDIGO fija cuál
```

$\text{primer_hit_que_nombra}(C, p, \text{visitadas})$:

```
16   $A \leftarrow \{c \in C \setminus \text{visitadas} : c \text{ menciona } p \text{ en forma COMPLETA}\}$    ▷ ancla
17   $M \leftarrow \{c \in C \setminus \text{visitadas} : c \text{ menciona } p \text{ en forma ABREVIADA}\}$    ▷ referencia
18  return  $\text{argmin}_{\{c \in (A \text{ si } A \neq \emptyset \text{ si no } M)\}} \text{posición\_primera\_mención}(c, p)$ 
```

Las cuatro líneas que son el aporte. La 3 y la 11 aplican la regla de creencias: una entidad nombrada se busca con el índice léxico, porque un vector denso codifica *de qué habla* un texto y sesenta documentos con la misma plantilla hablarían de lo mismo. La 10 tipa la salida del sensor antes de usarla: el modelo emite prosa, y esa prosa arrastraría la consulta a clasificarse mal. La 7 pone el largo bajo control del código. Y la 4 resuelve el ancla como cualquier otro salto, en vez de preguntárselo al modelo.

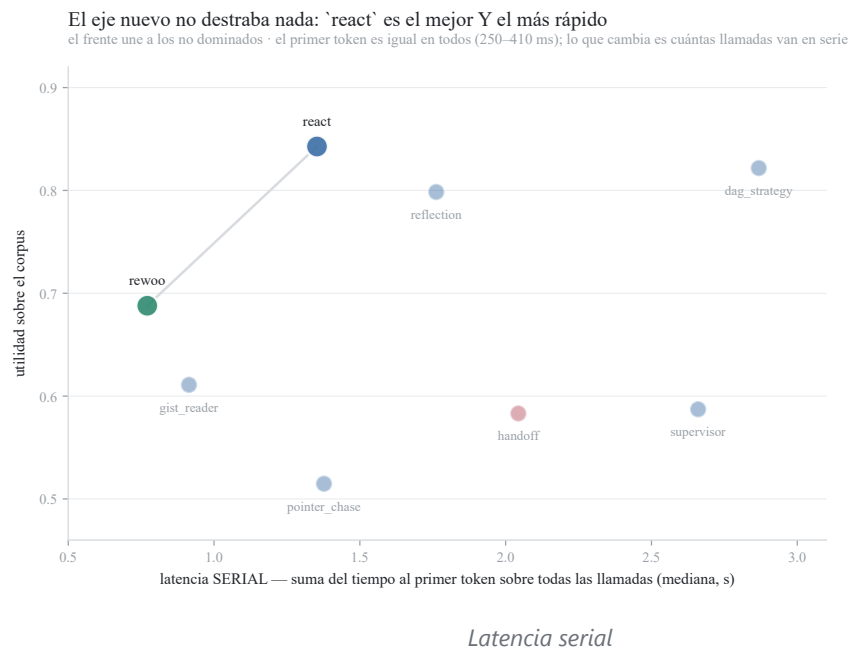
Las líneas 16–18 son el desempate, y son la parte que no es obvia: entre candidatos que mencionan la entidad, el que la nombra en forma **completa** es el que trata sobre ella y el que la abrevia sólo la referencia; con empate, gana el que la menciona **antes**. Ambas pruebas son predicados sobre el índice — devuelven un booleano o una posición, nunca texto— así que no cuestan un token.

Resultado: `pointer_chase` **pasa de 0,33 a 0,89 en C3**, empatando al mejor brazo de la celda. Y la corrección (4) prueba el punto por sí sola: **con la misma huella y los mismos resultados de búsqueda**, la réplica 0 elegía el ancla correcta y recorría la cadena entera ($u=1,000$) mientras las réplicas 1 y 2 elegían otra y sacaban 0,000. Una decisión de flujo en el sensor se lleva puesto el determinismo.

La hipótesis, y es falsable: sustituir una ramificación de control delegada al modelo por un medidor determinista sobre una señal del entorno mejora **la utilidad y el determinismo a la vez**. `passk` es la métrica que faltaba para medir el segundo efecto, y sin ella la mitad de la mejora era invisible.

Lo que queda declarado y no resuelto: la réplica que todavía falla **se abstiene** en vez de contestar mal, que es el comportamiento buscado y que el banco puntúa igual que un error.

7.2.5 La latencia serial varía 3,6× entre paradigmas y no entra en ninguna decisión



Hay **dos relojes** y confundirlos invalida el número. El tiempo de pared no sirve: el 63-67% de las filas de `react` y `dag_strategy` están por debajo de medio segundo porque son replays del caché en disco — eso mide cuánto tarda el banco en releer, no cuánto tarda el sistema en contestar. Lo que sí sirve viene del proveedor, en el objeto `usage` de cada respuesta, y por eso el caché lo conserva: es la latencia de la llamada que efectivamente se hizo.

brazo	u	primer token	latencia serial	u por segundo
react	0,843	265 ms	1,35 s	0,62
dag_strategy	0,822	410 ms	2,87 s	0,29
rewoo	0,677	304 ms	0,77 s	0,88
supervisor	0,577	360 ms	2,66 s	0,22

El primer token es prácticamente igual en todos —250 a 410 ms, es una llamada al mismo modelo—. Lo que cambia 3,6× es la latencia **serial**: la suma sobre todas las llamadas, o sea la parte que no se acelera con más tokens por segundo porque cada llamada espera a la anterior. Es la ley de costo `turn-driven` cobrada en tiempo del usuario en vez de en tokens, y es un eje que **ninguna decisión del ruteador mira hoy**.

Y no destraba nada, que es lo que hay que decir: `react` es a la vez el de mayor utilidad y el de menor latencia serial entre los contendientes. El único que compra tiempo es `rewoo` —1,8× más rápido— y cuesta 0,165 de utilidad. Es un intercambio explícito, no un almuerzo gratis: sólo lo compra quien tenga un techo de latencia declarado.

7.2.6 La varianza de trayectoria no crece con el número de ramificaciones

La sección anterior invita a una conjetura general, y **el registro la refuta en su forma ingenua**:

Si un brazo delega d decisiones de control al modelo, y cada una vuelve a salir igual entre réplicas con probabilidad q , entonces $\text{pass}^k \approx \text{pass}@1 \cdot q^d$. Más decisiones en el sensor $\Rightarrow$ menos determinismo, multiplicativamente.

Es contable: d se mide como iteraciones por celda, y pass^3 ya está. Sobre los ocho brazos del panel:

brazo	decisiones	pass@1	pass^3	q implícita
dag_strategy	8,9	0,822	0,703	0,983
supervisor	8,6	0,587	0,406	0,958
reflection	5,9	0,798	0,688	0,975
react	4,3	0,843	0,734	0,968
pointer_chase	3,8	0,515	0,359	0,909
rewoo	2,0	0,688	0,531	0,879
gist_reader	1,9	0,611	0,516	0,913

La correlación entre número de decisiones y caída de pass^3 es $r = -0,24$ con $n = 8$ — débil, y con el signo contrario al que la conjetura predice: los brazos con más decisiones pierden *menos*. La explicación que el propio dato sugiere es que una decisión no sólo agrega varianza sino también **una oportunidad de corregir**: un brazo adaptativo que dobla mal puede volver, y uno de dos llamadas no. Los dos efectos casi se cancelan en este corpus.

Lo que sí se sostiene, y es más útil que la conjetura original, son dos cosas:

1. q **está acotada lejos de 1 para todos**. El máximo es 0,983 (dag_strategy) y el mínimo 0,879 (rewoo). Ninguna arquitectura de las medidas recupera la reproducibilidad por decisión, así que **toda trayectoria con decisiones delegadas pierde determinismo**, y la pregunta no es si se pierde sino cuánto.

1. **Importa más CUÁL decisión se saca que CUÁNTAS**. La evidencia de esta sección no es correlacional sino una intervención: sacar **una sola** decisión —el ancla— llevó a pointer_chase de réplicas que discrepaban (1,000 / 0,000 / 0,000) a réplicas que coinciden, sin tocar las otras. Ocho puntos de correlación no compiten con eso.

El no-determinismo no se reparte por igual entre las decisiones de una trayectoria. Contar decisiones no predice; identificar **cuál** decide el resultado, sí.

Ésta es una limitación declarada del análisis, no un resultado: con ocho brazos y un corpus, lo correlacional no puede decidir casi nada. Lo que la sostiene es la intervención.

7.3 Por qué no hay premio de ruteo, aunque la interacción sea enorme

Éste es el resultado central del corpus, y es negativo con un mecanismo preciso. La forma en que suele argumentarse que rutear conviene —«hay mucha interacción entre tarea y método, así que elegir por tarea tiene que pagar»— **no se sostiene**, y este registro muestra dónde se rompe.

7.3.1 Hay interacción, y es grande

Descomponiendo $u(\text{tarea}, \text{brazo}) = \mu + \alpha(\text{tarea}) + \beta(\text{brazo}) + \gamma(\text{interacción}) + \epsilon$ sobre un panel de **59 tareas × 8 brazos**, que es el 76% de las 78 medidas. El criterio de recorte es mecánico y está en un solo lugar del código, no se elige por sección: **primero** las tareas que tienen al menos 7 brazos medidos —lo que impide que una tarea corrida por un experimento parcial redefina el universo—, **después** los brazos que cubren $\geq 95\%$ de éstas, y **al final** las tareas donde están todos. Los cuatro brazos que quedan afuera son los que la factibilidad poda en casi todas las celdas (`direct` , `streaming_scan` , `extract_compute`) más `graph_traverse` , que se declara infactible en el 56-59% de las celdas anchas.

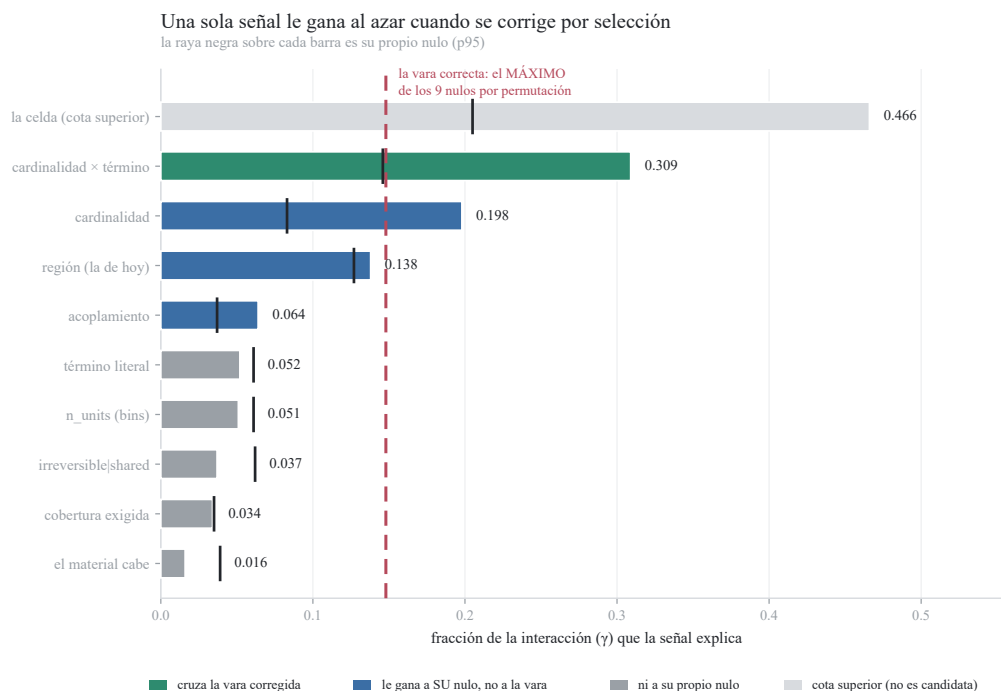
Se declara porque ésta es la sección que más insiste en corregir por selección, y un recorte sin criterio sería exactamente lo que audita en otros.

componente	varianza	%
α — dificultad de la tarea	0,0632	41%
β — calidad del brazo	0,0160	10%
γ — interacción	0,0736	48%
ϵ — ruido entre réplicas	0,0351	

El denominador, porque cambia el titular. Los porcentajes son sobre la varianza **explicada** ($\alpha + \beta + \gamma = 0,1528$). Contando ϵ , el total es $0,1879$ y γ pesa **39%**. Las dos lecturas son legítimas y la primera es la convención en descomposiciones de este tipo, pero un **48%** sin denominador declarado es un número que se lee más grande de lo que es.

y descontado el ruido da **0,0619**, con **señal/ruido 5,30**. Bajo el argumento habitual, correspondería rutear.

7.3.2 Una señal la explica y sobrevive la corrección por selección



Qué señal explica la interacción

La figura ordena diez señales por cuánto de γ explican, con **su propio nulo por permutación** dibujado como una raya negra sobre cada barra. Varias la superan — y esa comparación es exactamente la falacia que el nulo existía para evitar, porque **se probaron nueve candidatas y se eligió la mejor**. La vara correcta es la línea punteada: el **máximo de los nueve nulos en cada permutación**.

Sólo `cardinalidad × término literal` la cruza, con 0,309 y p corregido $< 0,001$, y captura **dos tercios del techo** que marca la **etiqueta de diseño** —el modo del que salió la tarea—, que está en el gráfico como **cota superior**, no como candidata: es la etiqueta de diseño del corpus, no se conoce al decidir, y ninguna señal real puede superarla.

7.3.3 El premio neto es negativo

$\text{var}(\gamma)$ **grande** $\neq$ **premio de ruteo grande**. El premio es $E[\max_p u] - \max_p E[u]$, y γ puede ser enorme porque los brazos **malos** son malos en lugares distintos. Esa estructura es real, es predecible, y **no vale nada**: nadie va a elegir el brazo que pierde por poco en vez del que pierde por mucho.

Lo único cobrable es la interacción **entre los brazos que competirían**. `gist_reader` aporta el 20% de $\text{var}(\gamma)$ y `rewoo` el 15%; los tres punteros, 6-7% cada uno. Restringido a esos tres —los que están a menos de 0,05 del mejor fijo—:

gamma real estimado	0,0046	señal/ruido	0,38
oráculo entre contendientes	0,932		
mejor fijo	0,875		
premio máximo	+0,058		
piso de ruido (bootstrap)	+0,065		
premio NETO	-0,008		

El piso de ruido, que es la cantidad de la que depende todo veredicto negativo de este paper.

Un oráculo toma un **máximo** sobre estimaciones ruidosas de utilidad por tarea, y un máximo sobre estimaciones ruidosas está sesgado hacia arriba: $E[\max_p \hat{u}_p] > \max_p E[u_p]$ **incluso cuando todos los paradigmas son idénticos**. Así que toda brecha de oráculo medida contiene una componente de ruido, y reportarla entera exagera el premio. La estimación es directa y no modelada: se toman las k réplicas del **mismo** paradigma sobre cada tarea, se las trata como si fueran k paradigmas distintos, y se calcula la brecha de oráculo sobre ellas. **Cada punto del resultado es ruido por construcción**, porque los «paradigmas» son la misma cosa. En el held-out se reporta el p95 sobre remuestreos de **tareas** —la tarea es la unidad de muestreo, no la fila—. Implementación en `app/metrics.py`; los deltas se computan sobre valores sin redondear, así que una resta de las cifras impresas puede diferir en la última posición.

Y **ninguna señal separa a los tres contendientes entre sí**: todas con $p > 0,29$. Sólo la `ce1da` lo logra ($p = 0,007$), y ésta no se conoce al decidir.

El 48% de interacción es real y vive **entre los brazos que nadie elegiría**. Reportar `var(y)` como evidencia de que rutear conviene mide la estructura equivocada.

7.3.4 El premio no reaparece agrupando paradigmas en familias

Una respuesta natural es que las señales quizá no separen individuos pero sí **grupos**: un ruteador que elige familia y después toma el más barato de la familia es otro ruteador, con otro premio. Se probó, declarando las familias **desde el código** —por cuándo el brazo decide su próxima llamada— y no desde el resultado:

familia	brazos	u media
<code>adaptativo</code>	<code>react</code> , <code>reflection</code>	0,818
<code>plan_fijo</code>	<code>dag_strategy</code> , <code>rewoo</code>	0,750
<code>canal_con_pérdida</code>	<code>gist_reader</code> , <code>handoff</code> , <code>pointer_chase</code> , <code>supervisor</code>	0,571

Valuar una familia por su **máximo** sería hacer trampa dos veces —le regala una elección por tarea que el ruteador de familias no puede hacer, y premia a la familia más numerosa por puro sesgo del máximo—, así que cada familia se representa por su brazo de mejor media, elegido **una vez** sobre todo el panel.

<code>adaptativo</code>	gana en 54 de 64 tareas	(84%)
<code>plan_fijo</code>	5 de 64	(8%)
<code>canal_con_pérdida</code>	5 de 64	(8%)
premio de rutear FAMILIAS	+0,079	(piso de ruido +0,065)
premio de rutear BRAZOS	+0,110	

Y hay que hacer la resta que el paper acaba de enseñar, porque los dos premios superan el piso: $+0,079 - 0,065 = +0,014$ para familias y $+0,110 - 0,065 = +0,045$ para brazos. Los dos netos son **positivos**, y aun así ninguno es cobrable — por una razón distinta de la de §7.3.3:

allá el premio no existía; **en este panel existe y no hay con qué agarrarlo**. El premio es lo que capturaría un oráculo, y un oráculo no es una política: **ninguna señal predice la familia ganadora**. La mejor, cardinalidad, da información mutua 0,092 y **no sobrevive la corrección por selección** ($p = 0,365$).

Un premio sin señal que lo prediga es una cota superior, no un resultado. La distinción importa porque las dos secciones concluyen lo mismo por caminos opuestos, y presentarlas juntas sin la resta invita al lector a pensar que nos contradecemos.

No falla porque las señales sean débiles: falla porque no hay frontera que cruzar. Una familia que gana el 84% de las veces no es un cluster que rutear — es un default.

7.3.5 Lo que este corpus sí premia

Sobre calidad la decisión no tiene premio. Sobre **costo a utilidad igualada**, el mismo registro da un ahorro grande con una pérdida dentro del ruido, y a diferencia del premio de calidad **sobrevive evaluarlo fuera de muestra** (leave-one-task-out):

señal	utilidad vs mejor fijo	ahorro
cardinalidad × término	-0,017	42%
región	-0,110	74%
n_units	+0,000	1%

Es un intercambio, no una mejora. Una medición previa sobre un panel más chico daba +0,008 de utilidad con 69% de ahorro —un almuerzo gratis— y **con el registro completo desapareció**. La pregunta «qué paradigma da la mejor respuesta» está agotada en este corpus; la pregunta «cuál es el más barato que da una respuesta indistinguible» no.

7.3.6 La región de una tarea depende del modelo que la sensó

El vocabulario de región tiene cinco ejes. Cuatro son COMPUTED —se calculan del texto de la pregunta y del material— y uno, el acoplamiento, es ELICITED: lo emite el modelo. Sobre las **26 tareas** medidas con dos familias de modelo:

eje de la región	procedencia	cambia entre modelos
cardinalidad	COMPUTED	0%
oráculo disponible	COMPUTED	0%
acoplamiento	ELICITED	27%
continuidad	COMPUTED	0%
término literal	COMPUTED	0%

Los cuatro ejes computados son idénticos en las 26 tareas; el elicitado cambia en 7. La misma pregunta, con el mismo material, cae en regiones distintas según qué modelo la sensó: few/oracle/loose/flat/no_lit con una familia y few/oracle/mixed/flat/no_lit con la otra.

Esto no es ruido de medición: es la clave de la política corriendo aguas abajo del sensor. Una política keyed en región hereda la estocasticidad del modelo en su propia clave — dos corridas del mismo request pueden buscar en dos filas distintas de la tabla. Es el mismo mecanismo de §7.2 aplicado un nivel más arriba: **una ramificación delegada al sensor, sólo que la ramificación es qué política se consulta.**

Una clave de política tiene que ser COMPUTED. Un eje elicitado describe lo que el modelo cree sobre la tarea, no la tarea, y usarlo para indexar convierte la tabla de decisión en una variable aleatoria.

Y da un mecanismo verificable para el fracaso del ruteo por región que §7.3.3 mide: parte de la señal que una política de este tipo puede aprender está en un eje que **no se reproduce entre modelos**, así que lo aprendido sobre un sensor no transfiere a otro aunque el corpus sea el mismo.

7.4 Qué es aprendible: la capacidad, no la identidad del paradigma

P15 se refutó mapeando ontología de la pregunta → **nombre de paradigma**: perdió $-0,087$ contra el mejor fijo. La sección anterior explica por qué ese premio no existía; ésta propone el eslabón que faltaba y **lo somete a la prueba que lo puede matar.**

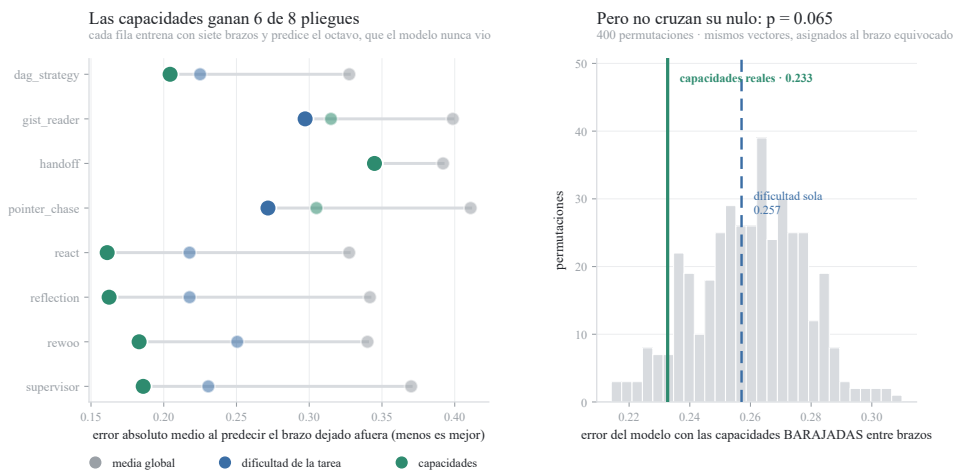
ontología de la pregunta → capacidades que EXIGE → brazos que las tienen

Diez capacidades declaradas **desde el código**, cada una con la medición que la justifica y con el sitio donde se ve: `PAYLOAD_COMPLETO`, `ADAPTA`, `COSTO_NO_ESCALA_CON_ALCANCE`, `LECTURA_SIN_PERDIDA`, `COBERTURA_GARANTIZADA`, `VERIFICA_Y_REPLANIFICA`, `RESOLVER_REFERENCIA`, `LARGO_GOBERNADO_POR_CODIGO`, `ELIGE_INDICE_POR_CONSULTA` y `ABSTIENE_SIN_PRUEBA`. Las cuatro últimas las destapó resolver la celda de cadenas acopladas, y ninguna es visible desde la taxonomía de control de flujo.

Y el catálogo encuentra un hueco sin correr nada: ningún brazo del plantel junta

`COBERTURA_GARANTIZADA` con `ABSTIENE_SIN_PRUEBA`, que es exactamente lo que una pregunta de ausencia exige. Eso es para lo que sirve declarar capacidades en vez de medir paradigmas — predice sobre un brazo que todavía no existe.

7.4.1 La prueba es dejar un BRAZO afuera, no una tarea



El EDA de capacidades

La asimetría es el punto: **un modelo con la identidad del paradigma no puede decir nada de un brazo que no vio** —no tiene parámetro para él, y es un límite estructural, no de ajuste—. Uno con capacidades sí, porque el brazo nuevo trae su vector declarado del código.

modelo	MAE al predecir el brazo dejado afuera
media global	0,364
dificultad de la tarea sola	0,257
capacidades	0,233
identidad del brazo (<i>viendo al brazo dejado afuera</i>)	0,339

Las capacidades ganan **6 de 8 pliegues** y bajan el error contra la dificultad de la tarea sola. La cuarta fila la puse creyendo que sería un techo y **no lo es**: sale peor que capacidades **pese a hacer trampa**, porque ignora α —la dificultad de la tarea, el 41% de la varianza—. Saber qué brazo es, sin saber qué pregunta es, predice mal. Ese fracaso es parte del argumento: **la identidad del paradigma no es una buena representación ni cuando se la deja mirar la respuesta.**

7.4.2 El efecto no cruza su nulo de capacidades barajadas

El nulo correcto no es la media global: es **barajar las capacidades entre brazos**. Mismos vectores, mismo número de rasgos, misma estructura, asignados al brazo equivocado. Si el modelo con capacidades reales no le gana a ése, lo que mide es la capacidad de **ajustar**, no la de **transferir**.

```
MAE con capacidades REALES      0,2327
MAE del nulo (barajadas)         media 0,2599 · p5 0,2299
p = 0,065
```

No cruza. Queda como resultado **sugestivo y no establecido**: ocho brazos son ocho puntos, y con esa n la prueba no puede decidir. Lo que la sentenciaría son más brazos, no más tareas.

Y la tabla ya tiene su propio contraejemplo anotado. EXIGE declara que una cadena acoplada pide RESOLVER_REFERENCIA y LARGO_GOBERNADO_POR_CODIGO, y bajo esa regla el único candidato es pointer_chase. Pero dag_strategy saca 0,89 en esa celda **sin ninguna de las dos**: llega por otra ruta, con VERIFICA_Y_REPLANIFICA para insistir y ABSTIENE_SIN_PRUEBA para no contestar cuando no llegó. A EXIGE le falta expresar **rutas alternativas** —hoy es una conjunción, y la realidad admite «A y B, o bien C y D»—. Se deja como conjunción y con el contraejemplo escrito, porque una tabla que se arregla sola para tapar su propio contraejemplo deja de ser falsable.

7.5 El consenso entre paradigmas es un verificador sin oráculo ni juez

Este resultado no se buscó. El banco corre ocho brazos sobre la misma pregunta y siempre los comparó **contra el oráculo**, nunca **entre sí** — y en el registro había ocho respuestas por tarea que nadie había mirado juntas.

¿El acuerdo entre paradigmas predice la corrección, sin oráculo y sin juez?

7.5.1 La curva

k brazos coinciden	celdas	P(la respuesta es correcta)
0	208	0,424
1	36	0,389
2	24	0,600
3	64	0,812
4	20	1,000
5	42	1,000
6	70	1,000
7	48	1,000

180 de 180 celdas correctas con $k \geq 4$, sobre igualdad exacta de la cadena normalizada. La curva tiene un **umbral** en 4 y no una pendiente suave. Dos salvedades que un **1,000** invita a saltar: sobre 180 casos la cota inferior de Wilson al 95% es **0,980**, así que lo afirmable es « $\geq 0,98$ »; y el 4 es **el punto donde la perfección aparece en este registro**, no un umbral derivado de nada.

7.5.2 Tres controles: no marca dificultad, no depende del largo, y no abarata

¿El acuerdo sólo marca «tarea fácil»? No. En las **mismas 27 tareas** donde existe consenso:

brazos DENTRO del consenso $n=180$ $u = 1,000$
brazos FUERA, en esas MISMAS tareas $n= 36$ $u = 0,100$

Sobre la misma pregunta, estar adentro o afuera del consenso es la diferencia entera: **discrimina dentro de la tarea**, no entre tareas. Si sólo marcara dificultad, los dos grupos rendirían igual.

¿Es un artefacto de comparar cadenas cortas? Tampoco. Una cuenta se compara fácil y una enumeración de cuatro ítems no, así que el efecto podría vivir sólo en **singular**. No lo hace:

cardinalidad	con consenso	sin consenso
aggregate	1,000 (n=34)	0,267
boolean	1,000 (n=41)	0,000
enumerative	1,000 (n=28)	0,555
singular	1,000 (n=77)	0,532

¿Sirve para abaratar? No, y se reporta igual. Es la lectura comercial obvia —comité barato, escalar sólo al discrepar— y se probaron las 56 cascadas de dos y tres brazos: **ninguna ahorra**. El comité se paga en todas las tareas y el brazo caro se paga igual en la mayoría, así que el total sube. **El consenso no es un ruteador barato.**

Algoritmo 2. El verificador, y qué le está permitido tocar de la base de creencias.

ALGORITMO 2 Verificación por consenso, sin oráculo ni juez

Entrada : pregunta q , plantel P , umbral k , base de creencias B

Salida : B con la credencia actualizada

```
1   $R \leftarrow \langle \text{ejecutar}(p, q) : p \in P \rangle$                                 ▷  $|P|$  trayectorias
2   $R \leftarrow \langle \text{normalizar}(r) : r \in R, r \neq \varepsilon \rangle$                 ▷ vacía ≠ abstención
3  for cada respuesta distinta  $a \in R$  do
4       $m(a) \leftarrow |\{ r \in R : r = a \}| - 1$                         ▷ cuántos coinciden
5   $a^* \leftarrow \text{argmax}_a m(a)$ 
6  if  $m(a^*) \geq k$  then
7       $B \leftarrow \text{asentar}(B, a^*, \text{procedencia} = \text{ELICITED},$         ▷ NO sube de nivel
8                                      $\text{credencia} = \text{calibración}[m(a^*)])$ 
9  else
10      $B \leftarrow \text{asentar}(B, a^*, \text{procedencia} = \text{ELICITED},$ 
11                                      $\text{credencia} = \text{calibración}[m(a^*)])$ 
12      $\text{marcar\_para\_verificación}(B, a^*)$                                 ▷ el dial decide si se paga
13 return  $B$ 
```

$\text{calibración} : m \mapsto P(\text{correcta} \mid m \text{ coinciden}), \text{ medida en §7.5.1}$

La línea 7 es la que hay que leer con cuidado, y dice lo contrario de lo que uno espera: la procedencia **no cambia**. El consenso mueve la credencia y nunca el nivel.

7.5.3 Qué es, y qué NO autoriza

Es un **detector de corrección de precisión total y cobertura parcial** —27 de 64 tareas—, que es exactamente la forma de una regla de abstención: no dice qué brazo usar, dice **cuándo no hace falta verificar**. Hasta este punto, la única forma de saber si una respuesta estaba bien era tener el gold.

Y hay una tentación que hay que cortar de raíz. La lectura natural es «si cuatro brazos coinciden, la creencia sube de nivel». **No.** La escalera de procedencia — ASSUMED < ELICITED < OBSERVED < COMPUTED — clasifica **cómo se obtuvo** algo, no **cuánta confianza** merece. Cuatro paradigmas de acuerdo siguen siendo el modelo hablando: **votar no toca el documento**, así que nada puede ascender a OBSERVED por consenso. Permitirlo sería exactamente la falla que la escalera existe para impedir — que una mayoría del sensor se promueva sola al rango de un hecho computado.

Lo que sí autoriza es mover la **credencia** dentro de ELICITED, que en el motor de decisión es un campo distinto del de procedencia. Y ahí la tabla de arriba **es la curva de calibración**: $0,42 \cdot 0,39 \cdot 0,60 \cdot 0,81 \cdot 1,00$. El módulo de creencias declara como riesgo abierto que «las credencias elicitedas pueden estar mal calibradas... hasta que existan datos de calibración». Éstos son datos de calibración.

procedencia = de dónde vino · credencia = cuánto se le cree. El consenso mueve la segunda y no puede tocar la primera, y confundirlas convierte un detector útil en un permiso para que el modelo se autoacredite.

7.5.4 El efecto se reproduce sobre una segunda familia de modelo

Los ocho brazos no son independientes: comparten modelo, corpus y recuperador. Hay dos mecanismos compatibles con la curva de §7.5.1, y sólo uno la hace utilizable:

- **convergencia de trayectorias** — estructuras de control distintas llegan al mismo lugar

cuando ese lugar es el correcto, y se dispersan cuando no. Sería una propiedad de los paradigmas, y debería reproducirse con otro modelo debajo

- **el mismo modelo repitiéndose** — ocho envoltorios alrededor del mismo sensor producen la misma salida por la misma razón, y el acuerdo no es evidencia de nada

Se registró la predicción P29 **antes de correr**, con los tres desenlaces escritos: $P(\text{correcta} \mid k \geq 4) \geq 0,90$ sería convergencia; $\leq 0,65$ —cerca de la tasa base— sería el mismo modelo; entre medio no distinguiría. Se corrió sobre **otra familia de modelo**, 23 tareas $\times$ 8 brazos $\times$ 1 réplica, 209 celdas, cero errores de infraestructura. El tamaño relativo hay que decirlo: son **374 filas contra las 3.519 de la familia principal**, un décimo, porque ese modelo cuesta diez veces la entrada — así que la réplica cubre un subconjunto de celdas y no la grilla.

segunda familia de modelo	celdas	P(correcta)	cobertura
$k \geq 4$	99	1,000	14 de 23 tareas
$k \geq 3$	115	1,000	18 de 23 tareas

1,000 contra un criterio de 0,90 : convergencia. El consenso se reproduce sobre otra familia, así que la explicación barata —que sea el mismo modelo repitiéndose— queda descartada. **Y el control dentro de la tarea también se reproduce:** donde hay consenso a $k \geq 3$, los brazos que quedan afuera sacan 0,272 ($n=29$) contra 1,000 de los que están adentro.

7.5.5 El umbral es del modelo; el fenómeno no

El umbral se movió, y **hacia abajo:** $k \geq 4$ en el primer modelo, $k \geq 3$ en el segundo. La guarda registrada en P29 anticipaba exactamente este caso —«si el umbral se mueve, eso ya es información»— y lo que dice es que **el número exacto de acuerdos es una propiedad del modelo y la señal no**. Sobre el modelo mejor hacen falta menos acuerdos para la misma precisión, que es lo que uno esperaría si la señal fuera convergencia y no coincidencia.

La cobertura también sube: **18 de 23 tareas (78%)** en el segundo modelo contra 27 de 64 (42%) en el primero. El detector es **más** útil sobre el modelo mejor, no menos — lo cual descarta la lectura de que el consenso sea una muleta para modelos flojos.

El umbral se compara sobre planteles del mismo tamaño, y esa condición no es opcional. k acuerdos no significan lo mismo sobre planteles distintos: $k=4$ sobre ocho paradigmas son 4 de 7 otros (57%), sobre doce son 4 de 11 (36%). Comparar el conteo crudo entre corridas con planteles distintos mide el denominador y no la señal. Las dos lecturas de esta sección están restringidas a los mismos ocho.

Lo que sigue sin estar probado. Dos familias no son la población de los modelos, y las dos comparten corpus y recuperador. Lo que se descartó es la explicación más barata; no se probó que valga para cualquier modelo ni para cualquier corpus.

7.6 Contra una ventana frontera: qué compra el arnés, y qué no

La objeción obvia a todo lo anterior es que un modelo grande con una ventana enorme lee todo el material de una vez y se ahorra la orquestación. **Sobre calidad, la objeción es correcta y los datos de este registro la respaldan:** el embudo de §7.2.1 muestra que toda representación intermedia más chica que el material es una pérdida, y una ventana grande es el caso extremo de *no tener canal con pérdida*. Medido: `direct` —leer todo en una sola llamada— saca **0,917**, el mejor del plantel donde corre.

El arnés no le compite ahí. Le compite en otros cuatro ejes, y conviene decirlos con número.

7.6.1 El precio

Una tarea de la banda ancha lleva **455.476 tokens** de material. Con los aranceles vigentes, y contando que por encima de 272.000 tokens de entrada se cobra tarifa larga **por el request entero**:

leer todo de una	USD por pregunta
ventana de 922k, gama media	0,18
ventana de 922k, gama alta	1,82
ventana de 922k, tope de gama	4,55
react sobre el modelo de la campaña (medido)	0,0216

211x. Y el escalón es un acantilado, no una pendiente: cruzar el umbral por un token duplica la tarifa del request completo.

7.6.2 Lo que no entra, y lo que no es un archivo

El corpus de este trabajo cabe en una ventana de 922k. **Un corpus de producción no**. Y ése es el caso fácil: el difícil es que **el material no siempre es un texto que se pueda pegar**. Un arnés extiende el alcance a lugares donde una ventana no llega por definición:

- **búsqueda externa** — un índice que cambia entre una pregunta y la siguiente, o que vive detrás de permisos por usuario. No hay ventana que contenga un índice vivo

- **corpus sin cota** — cuando el material crece con el negocio, «entra en la ventana» es una propiedad que vence. La ley de costo de §7.1.6 dice qué pasa cuando no entra

- **herramientas con efecto** — una escritura, una transacción, un ticket. Un modelo que lee no ejecuta, y lo que gobierna una acción irreversible es una precondition verificada, no un contexto grande

Y hay que tomarlo en cuenta con honestidad: nada de esto está medido en este trabajo. El corpus de este paper es estático, cabe, y no tiene herramientas con efecto. Son razones por las que un arnés existe, no resultados de este trabajo.

7.6.3 Los tres ejes donde una ventana no ayuda

Éstos sí están medidos, y son la parte que no depende del tamaño del modelo:

Abstención. El paradigma que gana la celda de cadenas acopladas lo hace con **12 correctas, 3 abstenciones y cero respuestas equivocadas**; los demás contestan igual y se equivocan. No gana por razonar mejor: gana porque cuando no llegó, lo dice. Saber *cuándo no contestar* es una decisión de flujo de control, y §7.2.4 muestra que sacarla del sensor sube la utilidad y el determinismo a la vez.

Procedencia. Un contexto grande no dice **de qué unidad salió un número**. El Teorema 2 —soundness del ensamblador— garantiza que si se emite, todo valor emitido está implicado por la base de creencias al piso pedido; eso exige una base de creencias tipada, no una ventana.

Determinismo. `pass^3` mide 17-34% de celdas inestables con temperatura cero y semilla fija, y esa varianza **no es del tamaño del modelo**: es la trayectoria componiéndola. Un modelo más grande con las mismas decisiones delegadas al sensor tiene el mismo problema.

El arnés no existe para ganarle a un modelo grande en calidad. Existe para que la respuesta cueste dos órdenes de magnitud menos, para alcanzar material que ninguna ventana contiene, y para poder decir **de dónde salió** y **cuándo no la hay** — que es lo único que un dial de garantía puede prometer.

La comparación directa no está corrida. No se midió un modelo de ventana frontera sobre las celdas anchas: el precio de §7.6.1 sale de los aranceles y del tamaño del material, y la calidad de `direct` sale de las celdas donde la factibilidad lo dejó correr. Es una inferencia bien apoyada, no una medición, y sería una corrida acotada —18 tareas de banda ancha, unos USD 82— que decidiría el punto de una vez.

7.7 Fuera de muestra: el corpus held-out completo

Todo lo anterior es **en muestra**. El criterio de éxito declarado del producto es otro —brecha de oráculo neta positiva sobre datos que el sistema nunca vio— y hasta este borrador **no existía como medición válida**: la única corrida del corpus held-out estaba en un archivo anterior al cambio de tokenizador, sin `analyzer`, sin huella y sin vocabulario de región estampados, o sea no replayable.

El held-out corre por el mismo camino de código que la campaña, con el corpus como argumento del runner. Es una condición y no un detalle de implementación: **un held-out medido con otro arnés no mide generalización, mide dos arneses**.

7.7.1 El premio neto es negativo en los tres estratos

26 tareas × 12 paradigmas × 3 réplicas, 927 filas, **48,5M tokens**, 135 minutos, **cero errores de infraestructura**. El rectángulo queda en **24 tareas × 8 paradigmas** —el 92% de las medidas— bajo el mismo criterio mecánico que el resto del paper.

paradigma	u
dag_strategy	0,802
react	0,777
reflection	0,722
supervisor	0,666
gist_reader	0,623
rewoo	0,613
pointer_chase	0,498
handoff	0,466

	base+w4	+ w16	+ w48
tareas del panel	12	18	24
oráculo por tarea	0,972	0,981	0,931
mejor fijo	0,833	0,813	0,802
brecha observada	+0,139	+0,168	+0,128
piso de ruido (p95)	+0,167	+0,187	+0,162
brecha NETA	-0,028	-0,019	-0,034

La brecha queda por debajo de su piso de ruido en los tres estratos, sobre 5, 20 y 60 unidades de alcance. Fuera de muestra el ruteo por calidad no tiene premio, que es el mismo veredicto de \$7.3 — -0,008 en muestra— por un camino independiente y sobre un corpus con otra mezcla de celdas.

El piso de ruido no decrece al agregar tareas, y eso es una propiedad del estimador. Un piso construido sobre la media decrece como $1/\sqrt{n}$; éste no es una media. Es $E[\max_p u] - \max_p E[u]$ bajo remuestreo —el **sesgo del máximo**— y crece con la dispersión entre paradigmas y con cuántos compiten. Cada ancho nuevo agrega dispersión, así que el piso se mueve con la composición del panel y no con su tamaño.

El piso de un premio de máximo se recomputa, no se proyecta. Cualquier cálculo de potencia que use la intuición de una media subestima el piso justo donde la comparación se decide.

7.7.2 El plantel no tiene un ganador estable, y eso es el resultado

Sobre el corpus completo los dos primeros quedan en 0,802 y 0,777, con una brecha de 0,025 contra un piso de 0,162. Sobre sub-paneles del mismo corpus el orden entre los punteros se permuta —tres paneles anidados dan tres punteros distintos— sin que ninguno se despegue.

No es que el mejor paradigma fijo cambie entre corpus: es que no hay uno. La afirmación «el paradigma X es el mejor» exige una separación que este registro no tiene, y un panel más chico no la produce —la fabrica, porque el máximo de pocas muestras es sesgado hacia arriba.

Es el mismo mecanismo de §7.3.3 mirado desde otro lado: donde los candidatos empatan, el máximo por tarea es el sesgo del máximo y no una elección mejor.

7.7.3 Costo de la corrida

estrato	tareas	material medio	celdas	tokens	tiempo
base + w4	14	91k	500	16,0M	75 min
w16	6	150k	216	13,7M	27 min
w48	6	451k	216	18,8M	33 min
total	26		932	48,5M	135 min

Las seis tareas de w48 cruzan el umbral de contexto largo, así que pagan **el doble de tarifa sobre el request entero**: es el acantilado de §7.7 aplicado a la propia corrida, y la razón de que el estrato más grande cueste 1,4× lo que su material sugiere.

8. Limitaciones y amenazas a la validez

Las amenazas se agrupan por el tipo de inferencia que ponen en riesgo. **Validez interna:** si el efecto medido es del mecanismo o del arreglo experimental. **Validez externa:** hasta dónde llega lo medido fuera de este corpus. **Validez de constructo:** si lo que se mide es lo que se dice medir. Ninguna toca la Proposición 5.4 ni los dos teoremas —esos son verificados por máquina y no dependen del corpus—; todas tocan las afirmaciones empíricas de §7.

8.1 Validez interna

El régimen en-ventana es casi tautológico, y §7.4 es la escapatoria — parcialmente recorrida. El corpus detrás de §7.1–7.3 tiene 16k tokens en su punto más ancho, así que leer-todo es correcto y lo más barato ahí. El régimen fuera-de-ventana ya está medido a 483k con `repeat = 3` (§7.4–7.5), y el claim de régimen a través de 135k/483k/1,27M descansa en el barrido de factibilidad a costo cero. Sigue abierto: no hay corrida completa a 1,27M (sólo su aritmética de factibilidad), dos celdas pesadas fuera de ventana todavía sin réplica completa (marcadas † en §7.4).

Ya existen réplicas, y el ruido es por celda. La réplica accidental del primer estudio (§7.3) y el `repeat = 3` del segundo coinciden: la reproducibilidad es por tarea — los lectores no dan vuelta nada, los que usan herramientas dan vuelta las celdas donde decide el retrieval, y el costo no fue reproducible en ninguna parte (dispersión media 2,05×, máxima 5,43× a calidad idéntica en el primer estudio; el mismo patrón de concentración en el segundo). Por eso todo delta de calidad se reporta con su conteo de flips por celda, nunca como media pelada. El piso de ruido formal por celda y la brecha de oráculo neta (`Study.noise_floor`, decisiones contra la brecha neta) se computan en el análisis final sobre la grilla completa — pendiente de las celdas en re-corrida.

La amenaza del sampleo no es la temperatura sino la combinación de `tools` con razonamiento. Medido sobre las tres familias disponibles: los deployments de razonamiento rechazan `temperature` de plano, y `terra` en su propio default es el **más** reproducible de los tres. La restricción viva es otra y es estructural: en la familia `5.6`, `tools` y un `reasoning_effort` distinto de `none` no se pueden combinar en Chat Completions, y la diferencia que eso hace es total — 0,000 contra 1,000 sobre la misma tarea. Así que toda fila de campaña corre con `reasoning_effort = none`, que es un régimen declarado y no un default.

8.2 Validez externa

Una rama de la propia partición no se pudo ejercitar, y es estructural. §5.2 parte el problema según `v`, la disponibilidad de un detector barato en runtime, y manda `v = 1` a una cascada y `v = 0` a un ruteador. **Todos los corpus de este registro caen del lado `v = 1`**, y no por elección: corregir sin juez significa corregir por coincidencia exacta, la coincidencia exacta necesita una respuesta de referencia, y el mismo campo se leía como detector de runtime. La regla de cascada dispara con prioridad más alta que la de selección, así que sobre dos corpus held-out **la regla de selección no disparó nunca** y los dos veredictos de ruteo son mediciones de la cascada. La forma general es una advertencia sobre toda una clase de experimento:

Un benchmark que establece corrección por coincidencia exacta contra una referencia **tiene un detector barato en cada tarea por construcción**, y por lo tanto no puede ejercitar la rama `v = 0` de su propia partición. Ser corregible implica ser verificable.

Si la selección paga donde verificar es genuinamente imposible queda **abierto**, y necesita un corpus cuya disponibilidad de detector se declare por tarea en vez de heredarse de la clave.

La transferencia entre mundos está medida, y es parcial. El mundo held-out con seed nueva — generado y verificado independientemente— ya corrió su test de transferencia registrado, P8. **Dos de sus cinco predicciones no transfieren**, así que por su propia regla de decisión registrada los veredictos por celda de más abajo son **corpus-locales**: afirmaciones sobre mundos seed-7, cada una con salvedad por mundo. La refutación no se apoya en las celdas que nadie resolvió — en las dos donde otro brazo llega a puntaje perfecto, el fallback general devuelve 0,667 y 0,000.

Un confundente de la varianza de trayectoria que este montaje no puede separar. Las celdas inestables se miden con `t = 0`, semilla fija y el mismo prompt, así que la varianza observada tiene dos fuentes posibles: la ramificación delegada al modelo, que es la que §7.2 estudia, y el **no-determinismo del stack de servicio** —agrupamiento dinámico de requests, orden de reducción en punto flotante, versión del kernel— que produce logits distintos para entradas idénticas sin que el cliente lo pueda ver. La Proposición 5.4 no depende de cuál domine: con `d(T) = 0` la trayectoria es fija cualquiera sea la fuente del jitter. Lo que sí depende es atribuir el 17–34% a la ramificación, y separarlo exige un servidor de un solo request por lote, que este montaje no tuvo.

Corpus sintético. El ground truth es exacto y se re-deriva de forma independiente, y los parámetros estructurales son diales y no esperanzas — pero la distribución de tareas reales sobre esos diales es desconocida. Hacen falta benchmarks públicos para validez externa y no se usan todavía.

La latencia no es comparable. El DAG de este banco corre secuencial donde correría en oleadas concurrentes; y una vez que los workers compiten, el wall-clock por fila deja de medir latencia. La calidad y los conteos de tokens siguen siendo exactos.

8.3 Validez de constructo

La superficie de acciones es arquitectura declarada y no está ejercitada. §5.5 filtra las acciones irreversibles con un piso de procedencia y §5.6 afirma la maquinaria sobre cuatro superficies. Una de esas cuatro nunca corrió. El inventario completo de herramientas, sobre todos los corpus y los quince paradigmas registrados, son doce:

qué hacen	cuáles
leen el mundo	<code>search</code> <code>keyword_search</code> <code>semantic_search</code> <code>read</code> <code>read_all</code>
escriben el estado del propio agente	<code>note</code> <code>notes</code> <code>plan</code> <code>advance</code> <code>post</code> <code>board</code>
leen su propia contabilidad	<code>coverage</code>

Ninguna de las doce cambia nada fuera del proceso. No se escribe un archivo, no se manda un mensaje, no se actualiza una fila. Seis tareas de setenta y ocho llevan `irreversible = True` y tres llevan `shared_writes = True`, y esas banderas sí levantan el dial de garantía — pero la tarea que etiquetan es

«decidí si esta relación debe escalarse para congelamiento; contestá 'escalate' o 'no escalation'», calificada por exact-match contra una clave. Es una clasificación sobre documentos con la etiqueta de una acción encima. No existe una herramienta que congele una cuenta, así que el piso que existe para filtrar lo irreversible nunca tuvo un acto irreversible que filtrar.

Resolverlo es otro experimento. El balance exacto: **§5 se afirma para agentes en general y está verificado como tal; §7–§8 se afirman para extracción de respuesta exacta sobre documentos y se midieron ahí; la superficie de acciones está diseñada, tipada, probada en unidad y sin medir.** Lo más parecido a evidencia sobre una capacidad que no es recuperación es el resultado del blackboard en §7.7, y es un nulo.

Las afirmaciones de novedad están verificadas como conjunciones, no como partes. Los dos trabajos ancla se leyeron completos el 2026-08-26: todos los números citados de Select-then-Solve verifican contra su cuerpo, y la cesión a SCL se sostiene. Las afirmaciones restantes — la consolidación de la política de control (§6.3), el descubrimiento de proposiciones con compuerta de procedencia, la abstención en el ruteo de paradigmas — sobreviven búsquedas fechadas sólo como conjunciones cuyos conjuntos individuales tienen cada uno un vecino publicado (§2.2, §2.4, §2.6). Un campo que se mueve así de rápido puede cerrar cualquiera de ellas en meses; las búsquedas están fechadas para que un lector pueda recorrerlas.

8.4 Reproducibilidad

Advertencia de reproducibilidad. Una semilla sola no fija un corpus. Cuando el algoritmo de generación cambió, la misma semilla produjo un mundo distinto; los manifiestos ahora estampan versión de generador y los resultados entre versiones no deben mezclarse.

9. Conclusión

Una sola máquina —sensor estocástico, creencia tipada, regla determinista, registro— y cuatro cosas que se pueden decir de ella con medida. No compiten ni se ordenan: miden la misma frontera —lo que el modelo emite contra lo que el código decide— desde cuatro lados.

	qué dice	evidencia
Confinamiento	dónde puede aparecer la varianza	absorber cuatro ramificaciones lleva un paradigma de 0,33 a 0,89 y hace coincidir sus réplicas; el ancla elegida por el modelo da 1,000 / 0,000 / 0,000 , elegida por el motor da 1,000 tres veces
Ruteo	qué se gana eligiendo	la interacción es el 48% de la varianza explicada (39% del total; S/R 5,30) y el premio neto es -0,008 en muestra, -0,034 held-out
Consenso	cuándo hace falta verificar	precisión total desde cuatro coincidencias, 180 de 180 celdas, replicado sobre una segunda familia de modelo
Capacidades	qué se predice de un paradigma que no existe	diez, declaradas desde el código, superan a la identidad del paradigma incluso cuando ésta ve la respuesta

Dos de los cuatro tienen mecanismo enunciado y es lo que transfiere fuera de este corpus. En confinamiento: la varianza no crece con el número de ramificaciones ($r = -0,24$), **importa cuál se absorbe**, y eso vuelve el diseño de un arnés un problema tratable —encontrar la ramificación que decide— en vez de una búsqueda sobre topologías. En ruteo: $\text{var}(y)$ grande no es premio grande porque la interacción vive entre los paradigmas que nadie elegiría, y el Teorema 1 es el instrumento que lo separa **antes** de construir un router.

Los otros dos son especificaciones más que observaciones. El consenso no abarata —las 56 cascadas probadas suben el costo— y no era para eso: es una **regla de abstención**, y su lugar es el dial de garantía. Y el catálogo de capacidades nombra un hueco sin correr nada: **ningún paradigma del plantel junta cobertura garantizada con abstención por prueba faltante**, que es lo que una pregunta de ausencia exige.

9.1 Lo que sigue

Cinco de las seis superficies plásticas siguen sin corpus que las mire. El ruteo es la única que éste permite. Las dos que cambian lo que el sistema *promete* —piso de garantía por estadísticas de rechazo, calibración por proposición— gobiernan la abstención y no la elección: piden un corpus donde la evidencia sea insuficiente en algunas tareas y suficiente en otras.

El nulo de capacidades da $p = 0,065$, y lo cruzan más paradigmas, no más tareas. Ocho puntos no deciden una transferencia; ampliar el plantel es concreto y barato.

Las otras tres superficies del motor —contenido, datos, acciones, gobierno— son el mismo enunciado del Teorema 2 sobre otro tipo de proposición.

Y una corrección gratis para cómo se evalúan agentes. Un banco que reporta `pass@1` sin `pass^k` no distingue un sistema que acierta de uno con el que se puede contar, y la diferencia llega a `0,196` en este registro. Uno que reporta varianza de interacción como evidencia de que rutear conviene mide la estructura equivocada.

Apéndice A — Artefactos

artefacto	contenido
el control nulo	el andamiaje por prompt se conserva en el registro y no se ejecuta nunca : dominado por <code>direct</code> en toda celda medida, a igual utilidad y jamás más barato. Es la evidencia de que andamiar por fraseo no compra nada, no un brazo
<code>PATTERNS.md</code>	catálogo de patrones: 10 estructurales, 4 de control, 15 anti-patrones, con aplicabilidad enunciada sobre el vector de features
<code>ANALYSIS.md</code>	el análisis de fallas de §8 completo, por paradigma y por celda
<code>GATE.md</code>	ocho criterios binarios de publicación y su veredicto actual
<code>PLAN.md</code>	historia de revisiones de la tesis, incluidos dos encuadres superados y por qué
<code>D:\Apps\MAPO\lab</code>	el harness: 15 paradigmas registrados , de los cuales 12 corren la campaña, 9 brazos de recuperación implementados de los cuales 2 corrieron alguna vez , 4 superficies de herramientas, 4 niveles de garantía, 12 herramientas, generador de corpus con verificador independiente, 573 aserciones chequeadas por máquina (521 + 52 en dos suites)

Siete de los quince anti-patrones del catálogo son errores cometidos y medidos en este trabajo, dos de ellos contra predicciones registradas de antemano.

Referencias

Los trabajos se identifican por su identificador persistente. Los marcados con ★ se leyeron completos en la fecha indicada; de los demás se verificó lo que este paper les atribuye, y donde el original no aporta un título corto se lo cita por identificador con la atribución que el cuerpo le hace.

Búsqueda y selección de workflows

Select-then-Solve. arXiv:2604.06753 — selección de paradigma por tarea; ★ leído completo el 2026-08-26. · **FlowBank.** arXiv:2606.11290 — portafolio offline, selección por consulta. · **TRACE-Router.** arXiv:2607.22465 — ruteo a granularidad de traza de tarea. · **Uno-Orchestra.** arXiv:2605.05007 — política conjunta de descomposición y despacho. · **Optimización de workflows como código,** con MCTS sobre operadores: arXiv:2410.10762. · **Query Optimization for Agentic Query Execution.** arXiv:2606.03152 — la analogía con la optimización clásica de consultas. · arXiv:2608.00106 — ruteo sobre features textuales, con *confidence gate* declarado como trabajo futuro sin evaluar.

Gobernanza simbólica y motores de creencias

★ **Jaime y Errecalde (2026).** **MINERVA/HADD.** Zenodo 10.5281/zenodo.20003407 (leído 2026-08-26); compuerta epistémica de admisión (EVR): Zenodo 10.5281/zenodo.19791686. **Es la base de la que §6.2 toma sus invariantes.** · **SCL / Soft Symbolic Control.** *Structured Cognitive Loop.* arXiv:2511.17673.

- **Nous.** arXiv:2606.22030 — confiabilidad acotada por la procedencia del canal. · **MemIR.**

arXiv:2605.25869 — memoria tipada por procedencia. · **Eywa.** arXiv:2605.30771 — promoción de hechos tras validadores. · **HEP.** arXiv:2607.09195 — evolución auditable de hipótesis. · arXiv:2607.01507 — diagnóstico empírico del jardín de senderos que se bifurcan. · arXiv:2603.27299 — política determinista, auditable y versionada, *escrita por humanos*. · **ProvenanceGuard.** arXiv:2607.01236 — pisos de procedencia sobre acciones. · **Contratos de delegación e identidad atestiguada.** arXiv:2603.18043. · arXiv:2605.26667 — atribución de error por registro de operaciones intermedias.

Aprendizaje sin actualizar pesos

Kintsugi. *Learning Policies by Repairing Executable Knowledge Bases.* arXiv:2605.09487 — **establece el paso de instalación filtrada por verificador** que §2.10 le cede. · **EnvProbe.** *Ask the World Before Acting: Budgeted Environment Probing for World-Model Calibration.* arXiv:2606.31422. · **SCM y afines,** consolidación de inspiración biológica: arXiv:2604.20943, arXiv:2605.26099. · arXiv:2606.10457 — destilación de control desde trazas de expertos a reglas reinyectadas *como texto de prompt*.

Predicción selectiva, deferral y evaluación

Chow (1970), *On optimum recognition error and reject tradeoff*. · Mozannar y Sontag, deferral con surrogate consistente [PMLR v119]; Verma y Nalisnick, deferral one-vs-all calibrado: arXiv:2202.03673. · arXiv:2605.07805 — descomposición de incertidumbre que unifica abstención y ruteo con garantías *distribution-free*. · **eDAct.** arXiv:2604.07036 — deferral por umbral calibrado de incertidumbre. · arXiv:2606.19544 — confiabilidad sin validez en jueces LLM, y sobre- estimación por acuerdo crudo. ·

arXiv:2512.15751. · **Survey.** *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents.* arXiv:2606.04990.

Métrica y linaje clásico

tau2-bench. Sierra, 2025. github.com/sierra-research/tau2-bench — origen de `pass^k`.

El linaje de §2.9. Alchourrón, Gärdenfors y Makinson (1985), *On the logic of theory change*. — Doyle (1979), *A truth maintenance system*. — de Kleer (1986), *An assumption-based TMS*. — Buneman, Khanna y Tan (2001), *Why and where: a characterization of data provenance*. — Green, Karvounarakis y Tannen (2007), *Provenance semirings*. — Rao y Georgeff (1995), *BDI agents: from theory to practice*. — Laird, Newell y Rosenbloom (1987), *SOAR: an architecture for general intelligence*. — Hebb (1949), *The Organization of Behavior*. — Dung (1995), *On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games*.

Marco normativo. Reglamento (UE) 2024/1689 de Inteligencia Artificial, arts. 12 y 14. — Reglamento (UE) 2016/679 (RGPD), art. 22.